COSC419/519:DEEP LEARNING

Training Neural Networks

Presented by: Omar Abdelaziz
Instructor: Dr. Mohamed Shehata
Computer Science
University of British Columbia

OKANAGAN

Adapted from Fei-Fei Li, Ehsan Adeli, Zane Durante

Overview

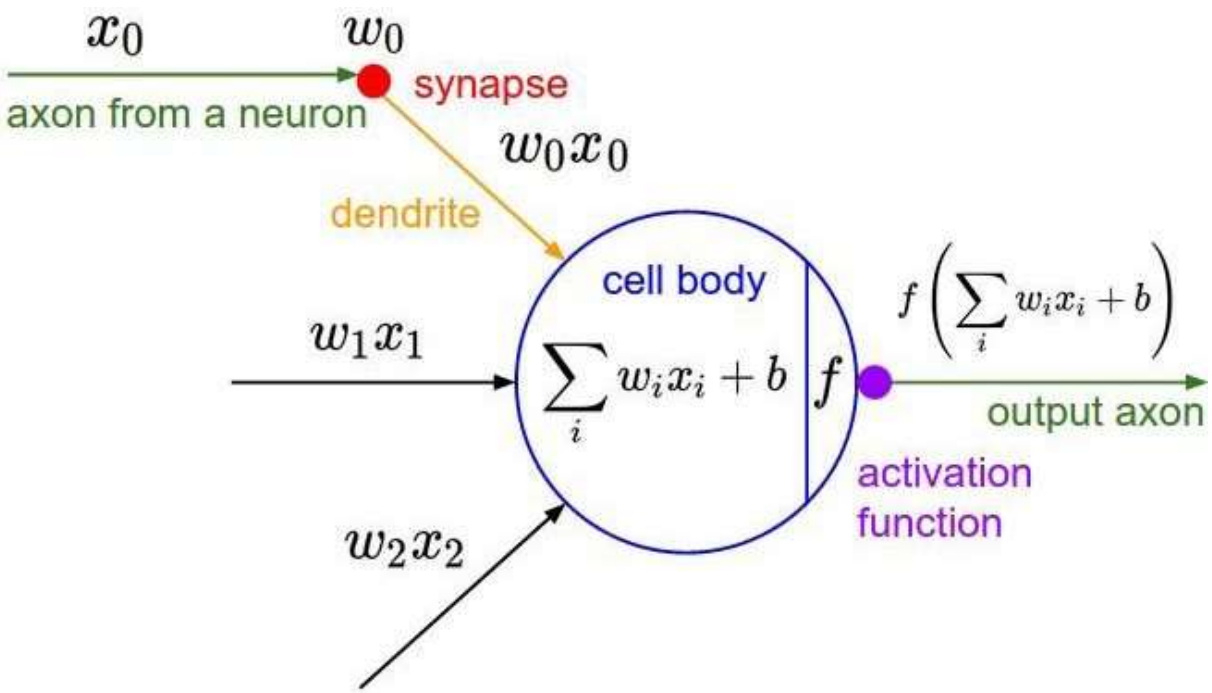
1. **One time set up**: activation functions, weight initialization, regularization

2. **Training dynamics**: hyperparameter optimization

3. **Evaluation**: transfer learning

Activation Functions

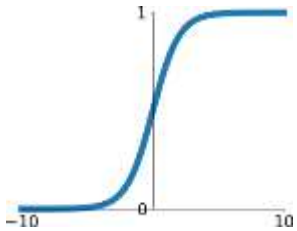
Activation Functions



Activation Functions

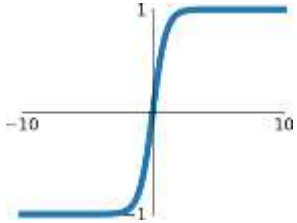
Sigmoid

$\sigma(x) = \frac{1}{1+e^{-x}}$



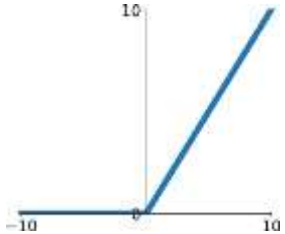
tanh

$\tanh(x)$



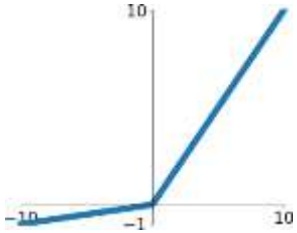
ReLU

$\max(0, x)$



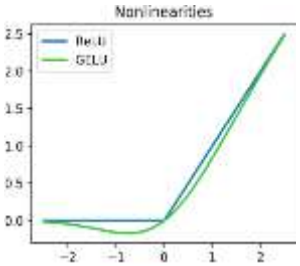
Leaky ReLU

$\max(0.1x, x)$



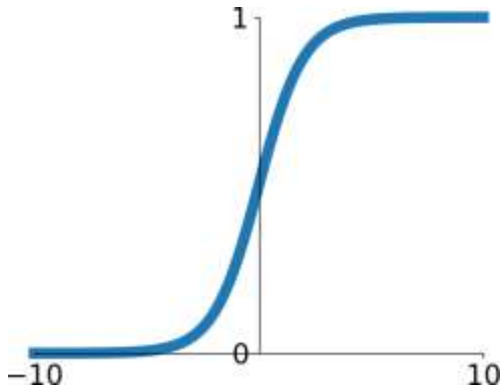
GELU

x*Φ(x)



Activation Functions

Activation F: $\sigma(x) = 1/(1 + e^{-x})$

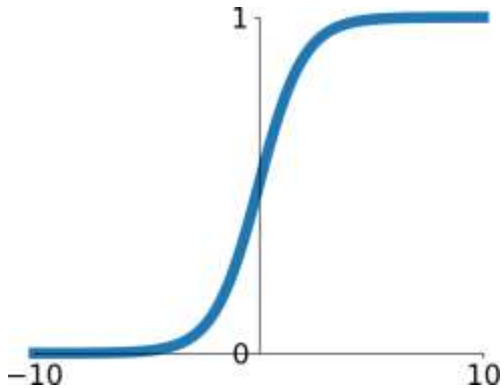


Sigmoid

- Squashes numbers to range [0,1]
- Historically popular since they have nice interpretation as a saturating “firing rate” of a neuron

Activation Functions

Activation F: $\sigma(x) = 1/(1 + e^{-x})$

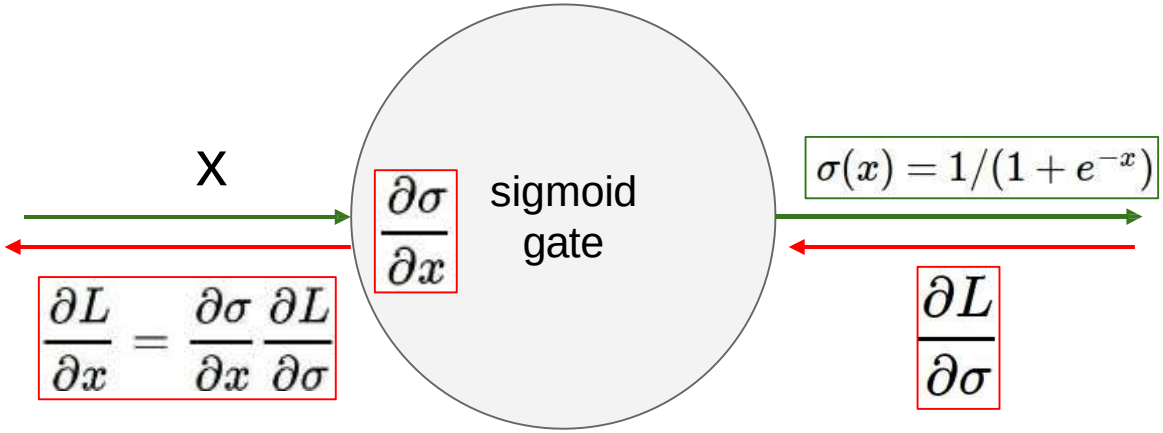


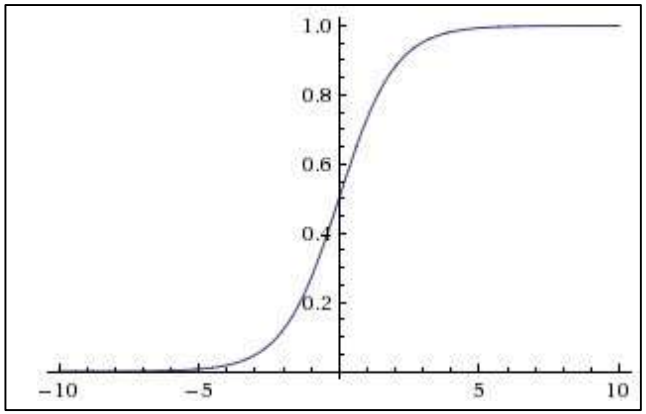
Sigmoid

- Squashes numbers to range [0,1]
- Historically popular since they have nice interpretation as a saturating “firing rate” of a neuron

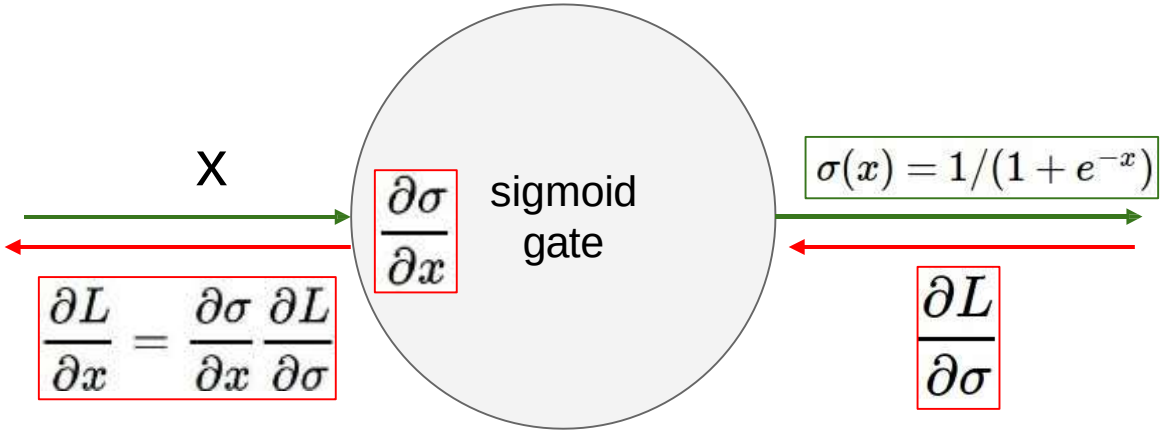
Problems:

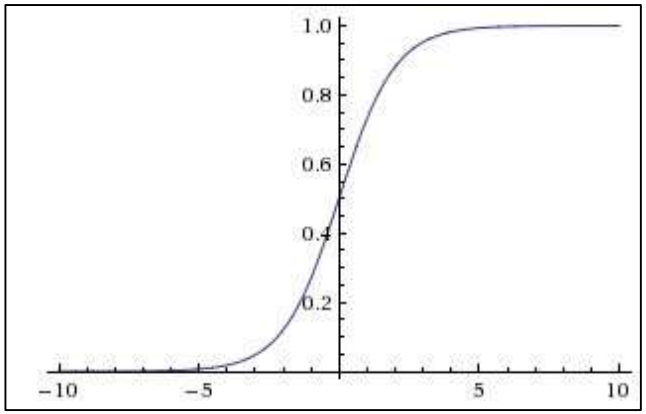
1. Saturated neurons “kill” the gradients





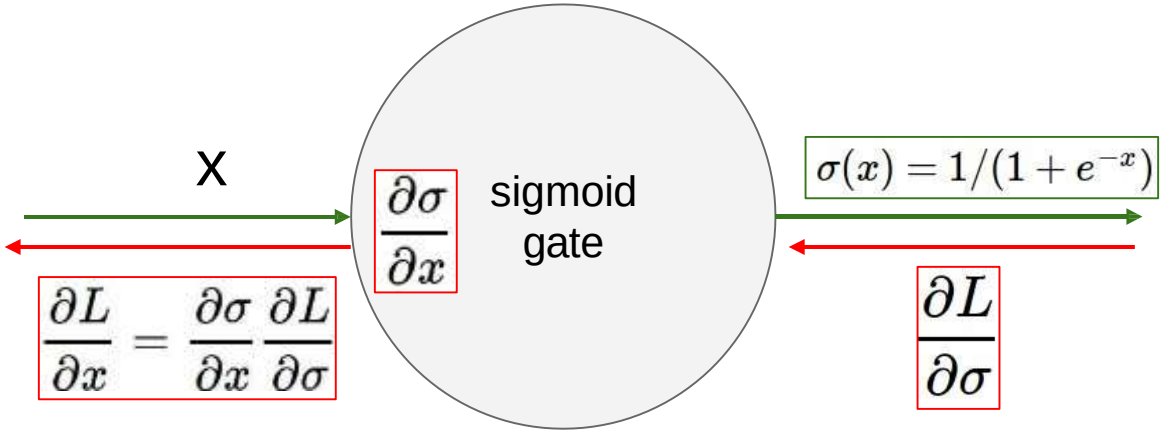
$$\frac{\partial \sigma(x)}{\partial x} = \sigma(x)\,(1 - \sigma(x))$$

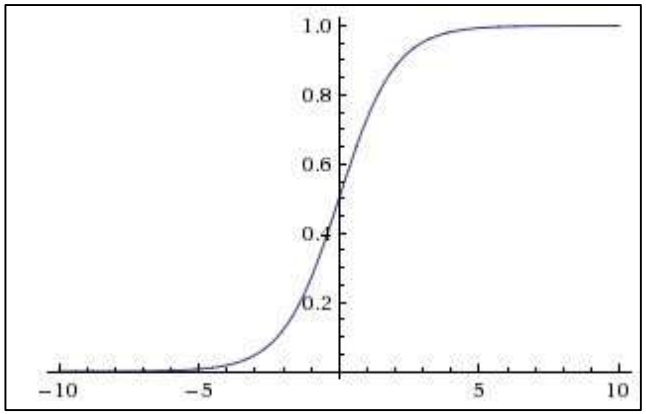




What happens when x = -10?

$$\frac{\partial \sigma(x)}{\partial x} = \sigma(x)\,(1 - \sigma(x))$$



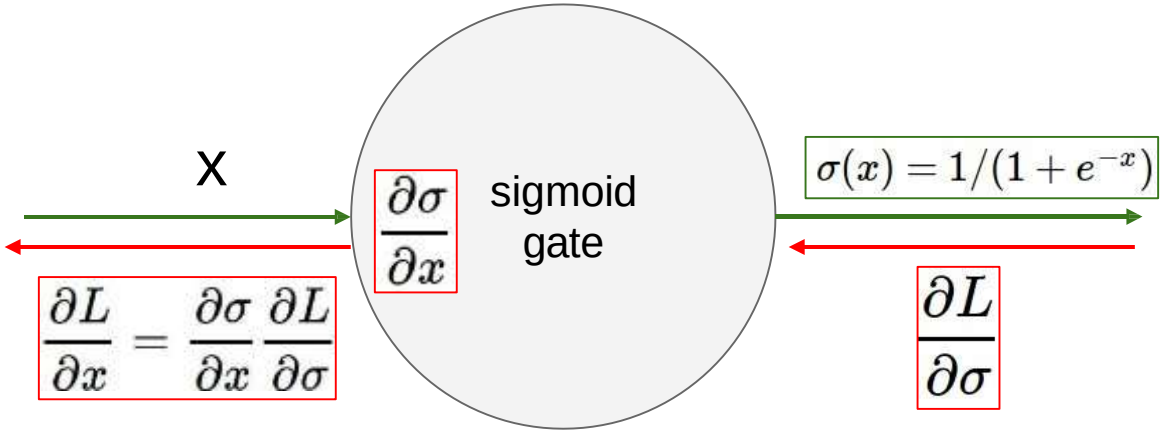


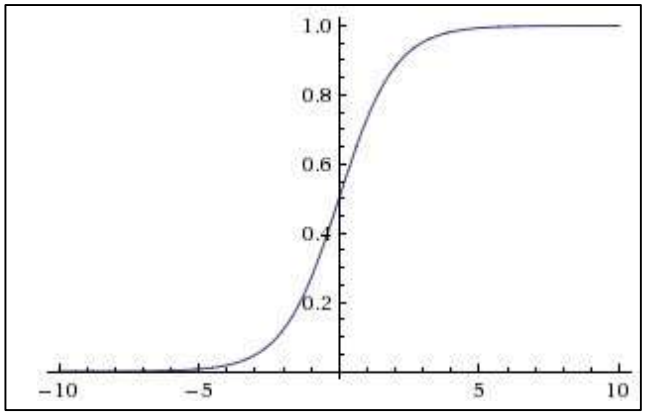
$$\frac{\partial \sigma(x)}{\partial x} = \sigma(x)\,(1 - \sigma(x)\,)$$

What happens when x = -10?

$$\sigma(x) = \sim 0$$

$$\frac{\partial \sigma(x)}{\partial x} = \sigma(x)\,(1 - \sigma(x)\,) = 0(1-0) = 0$$



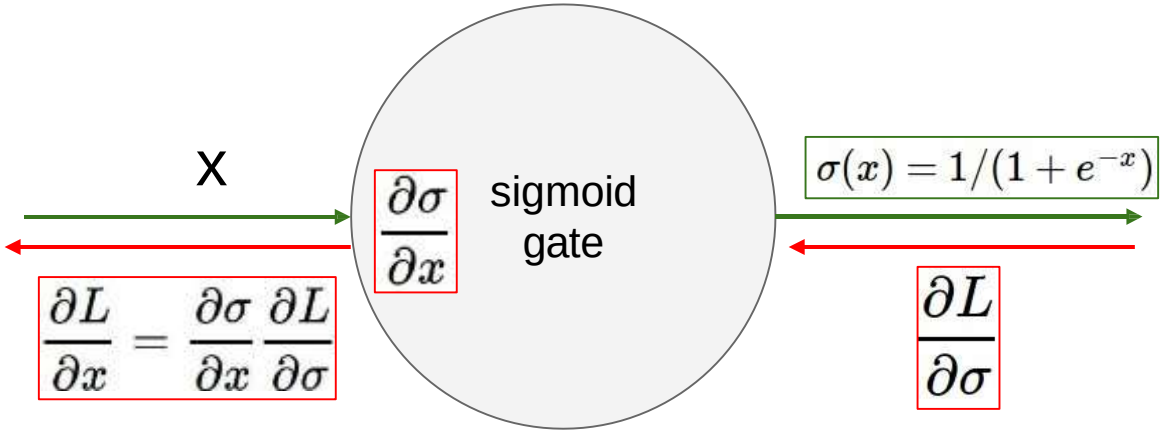


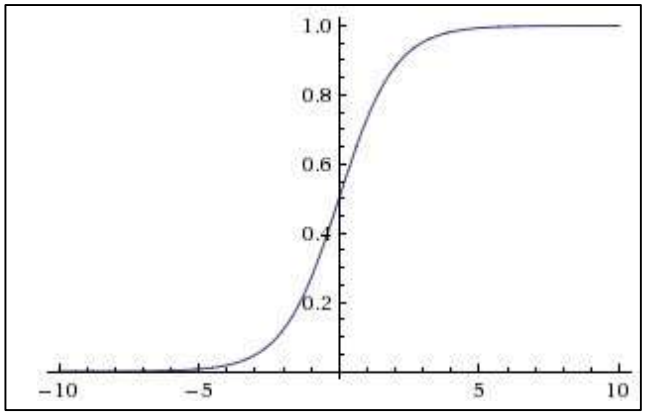
What happens when x = -10?

What happens when x = 10?

$$\frac{\partial \sigma(x)}{\partial x} = \sigma(x)\,(1 - \sigma(x))$$

$$\sigma(x) = \sim 1 \qquad \frac{\partial \sigma(x)}{\partial x} = \sigma(x)\,(1 - \sigma(x)) = 1(1-1) = 0$$

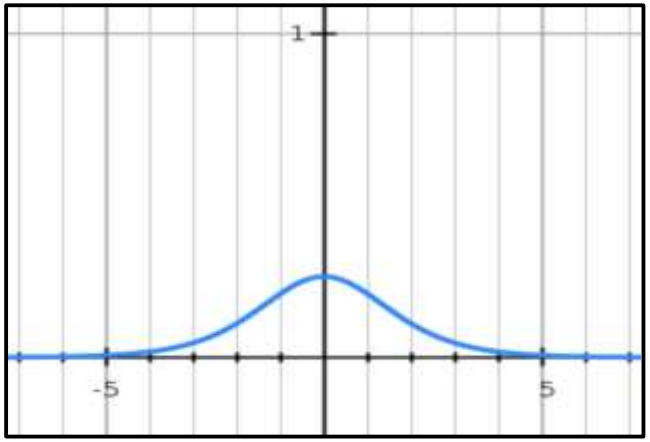


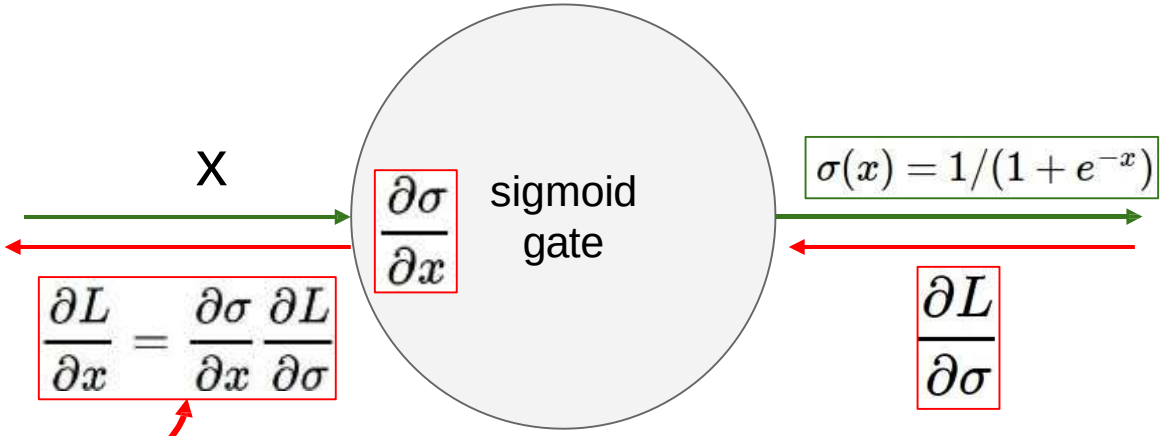


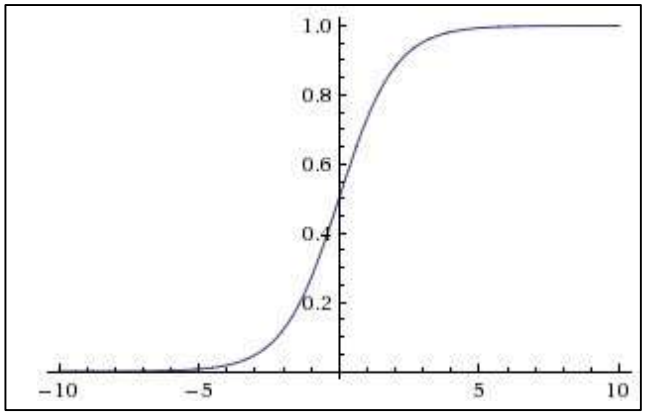
What happens when x = -10?

What happens when x = 10?

$$\frac{\partial \sigma(x)}{\partial x} = \sigma(x)\,(1 - \sigma(x))$$





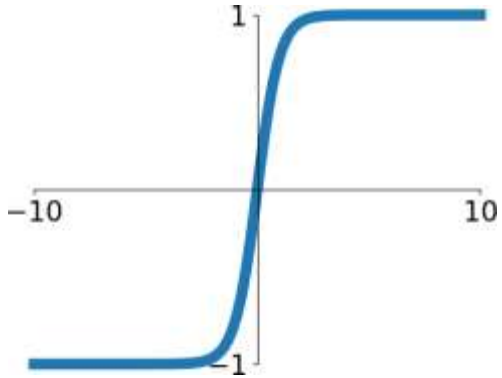


$$\frac{\partial \sigma(x)}{\partial x} = \sigma(x)\,(1 - \sigma(x))$$

Why is this a problem?

If all the gradients flowing back will be zero and weights will never change

Activation Functions



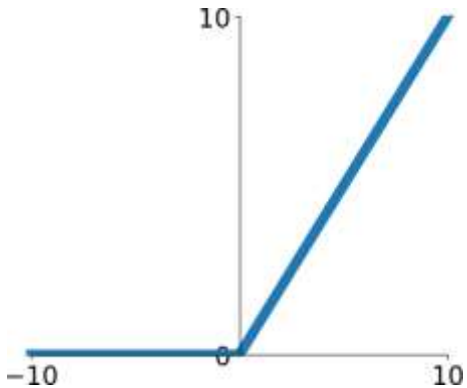
tanh(x)

- Squashes numbers to range [-1,1]
- zero centered (nice)
- still kills gradients when saturated

[LeCun et al., 1991]

Activation Functions

- **Computes f(x) = max(0,x)**



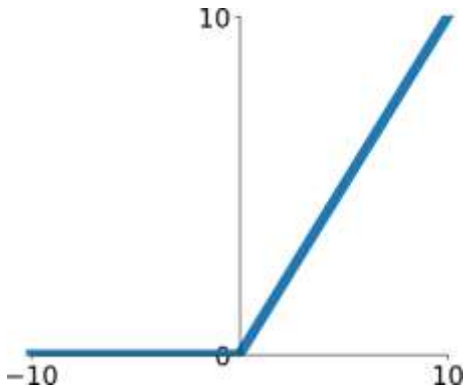
ReLU
(Rectified Linear Unit)

- Does not saturate (in +region)
- Very computationally efficient
- Converges much faster than sigmoid/tanh in practice (e.g. 6x)

[Krizhevsky et al., 2012]

Activation Functions

- Computes f(x) = max(0,x)



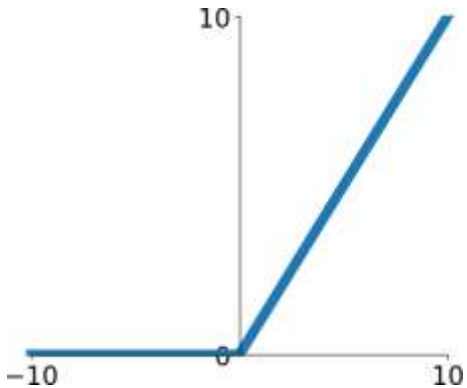
ReLU
(Rectified Linear Unit)

- Does not saturate (in +region)
- Very computationally efficient
- Converges much faster than sigmoid/tanh in practice (e.g. 6x)

- Not zero-centered output

Activation Functions

- Computes f(x) = max(0,x)

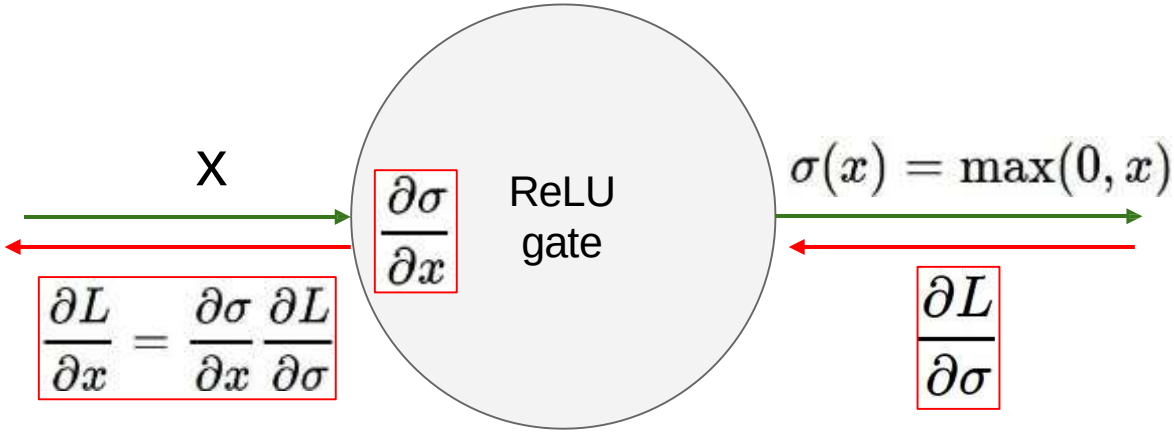


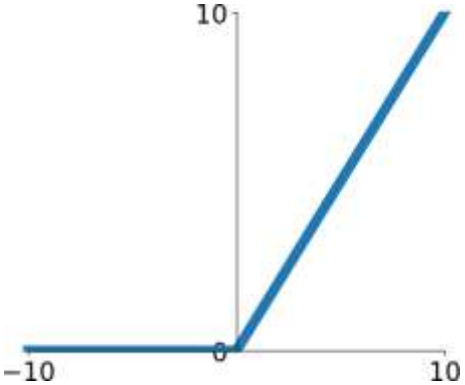
ReLU
(Rectified Linear Unit)

- Does not saturate (in +region)
- Very computationally efficient
- Converges much faster than sigmoid/tanh in practice (e.g. 6x)

- Not zero-centered output

hint: what is the gradient when x < 0?

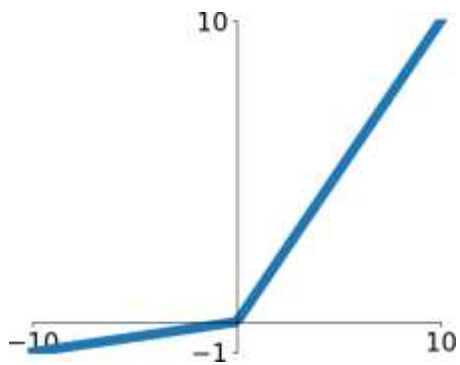




What happens when x = -10?
What happens when x = 10?

Dying ReLU problem: some neurons output zero for all inputs and stop learning because their gradients become zero. This typically happens when the input to the ReLU activation function is always negative, leading to zero output and no gradient updates during backpropagation

Activation Functions



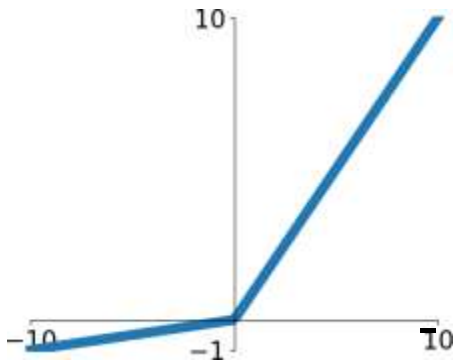
Leaky ReLU

$$f(x) = \max(0.01x, x)$$

[Mass et al., 2013]
[He et al., 2015]

-Does not saturate
-Computationally efficient
-Converges much faster than sigmoid/tanh in practice! (e.g. 6x)
-will not "die".

Activation Functions



Φ(x)

Leaky ReLU

$$f(x) = \max(0.01x, x)$$

[Mass et al., 2013]
[He et al., 2015]

-Does not saturate
-Computationally efficient
-Converges much faster than sigmoid/tanh in practice! (e.g. 6x)
-**will not "die".**

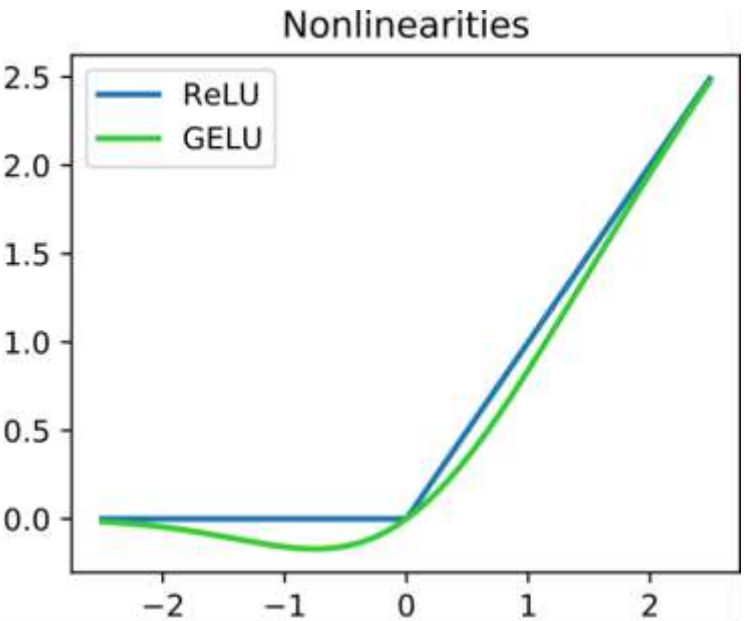
Parametric Rectifier (PReLU)

$$f(x) = \max(\alpha x, x)$$

backprop into α (parameter)

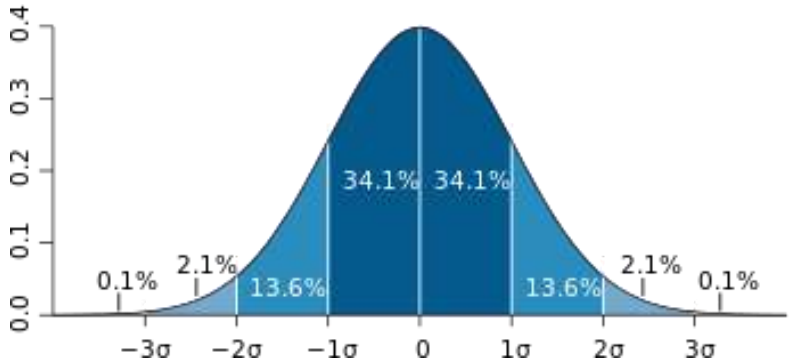
Activation Functions

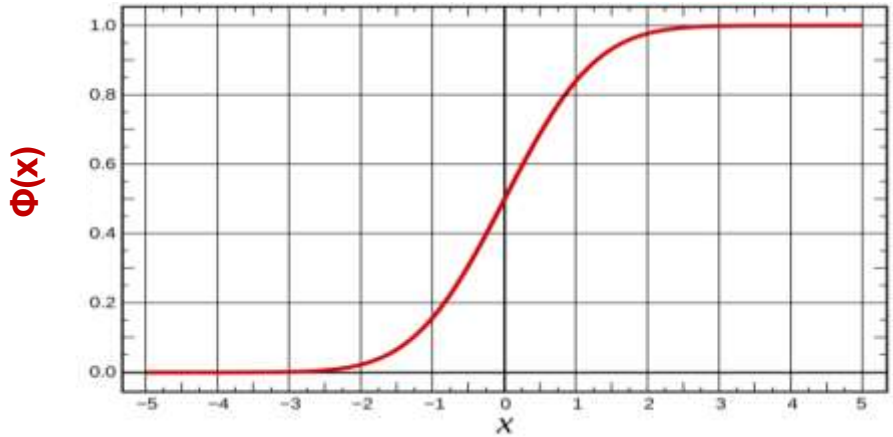
[Hendrycks et al., 2016]



Source: https://en.m.wikipedia.org/wiki/File:ReLU_and_GELU.svg

- Computes **f(x) = x*Φ(x)**



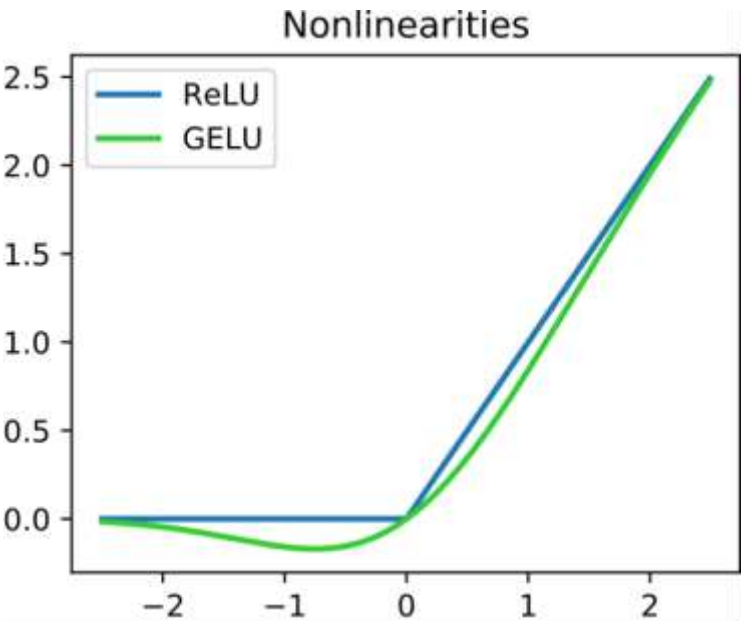


GELU
(Gaussian Error Linear Unit)

Sources: https://en.wikipedia.org/wiki/Normal_distribution, https://en.m.wikipedia.org/wiki/File:Cumulative_distribution_function_for_normal_distribution,_mean_0_and_sd_1.png

Activation Functions

[Hendrycks et al., 2016]



Source: https://en.m.wikipedia.org/wiki/File:ReLU_and_GELU.svg

- Computes **f(x) = x*Φ(x)**
- Very nice behavior around 0
- Smoothness facilitates training in practice
- Higher computational cost than ReLU
- Large negative values can still have gradient ➔ 0

GELU
(Gaussian Error Linear Unit)

TLDR: In practice:

- Use ReLU. Be careful with your learning rates
- Try out Leaky ReLU / PReLU/ GeLU
  - To squeeze out some marginal gains
- Don’t use sigmoid or tanh

Code: Activation Functions in PyTorch

PYTHON / PYTORCH

```
import torch
import torch.nn as nn

# Defining activation functions
relu = nn.ReLU()
leaky_relu = nn.LeakyReLU(negative_slope=0.01)
gelu = nn.GELU()

# Using in a model
class SimpleNet(nn.Module):
    def __init__(self, input_dim, hidden_dim, num_classes):
        super().__init__()
        self.fc1 = nn.Linear(input_dim, hidden_dim)
        self.act = nn.GELU()           # Recommended default
        self.fc2 = nn.Linear(hidden_dim, num_classes)

    def forward(self, x):
        x = self.act(self.fc1(x))      # Apply activation after
linear
        return self.fc2(x)
```

Recommendations

Default choice: ReLU

Fast, simple, works well for most CNNs. Set lr carefully.

For transformers: GELU

Smooth gradient near zero, used in BERT, GPT, ViT.

Dying neurons? Leaky ReLU

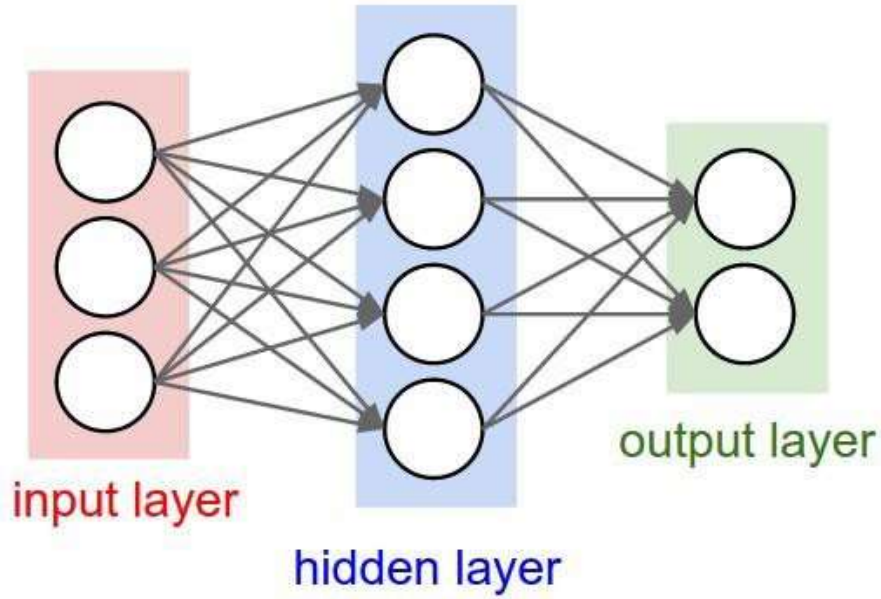
If many dead units (output always 0), switch to LeakyReLU or PReLU.

Avoid: Sigmoid / tanh

Vanishing gradients in deep nets. Only use in output layers (e.g. sigmoid for binary classification).

Weight Initialization

Q: what happens when W = constant init is used?



First idea: Small random numbers (gaussian with zero mean and 1e-2 standard deviation)

```
W = 0.01 * np.random.randn(Din, Dout)
```

First idea: Small random numbers (gaussian with zero mean and 1e-2 standard deviation)

```
W = 0.01 * np.random.randn(Din, Dout)
```

Works ~okay for small networks, but problems with deeper networks.

Weight Initialization: Activation statistics

```
dims = [4096] * 7
hs = []
x = np.random.randn(16, dims[0])
for Din, Dout in zip(dims[:-1], dims[1:]):
    W = 0.01 * np.random.randn(Din, Dout)
    x = np.tanh(x.dot(W))
    hs.append(x)
```

Forward pass for a 6-layer net with hidden size 4096

What will happen to the activations for the last layer?

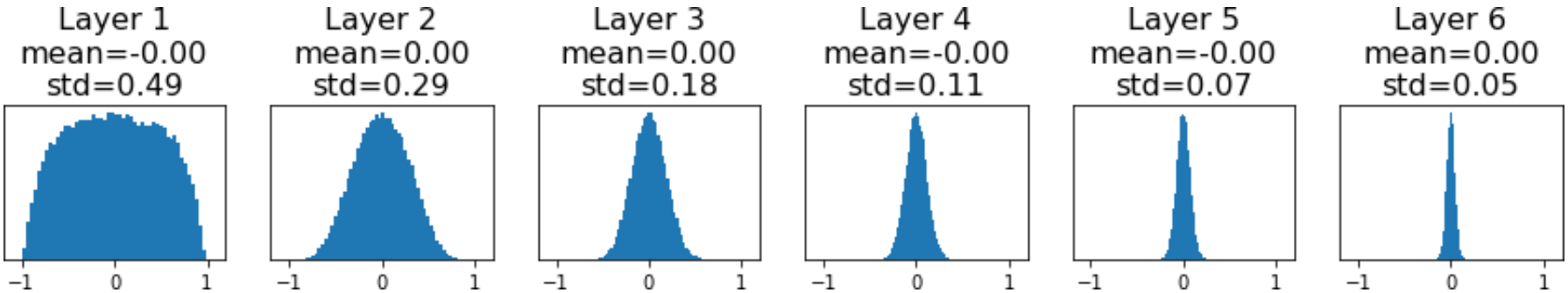
Weight Initialization: Activation statistics

```
dims = [4096] * 7
hs = []
x = np.random.randn(16, dims[0])
for Din, Dout in zip(dims[:-1], dims[1:]):
    W = 0.01 * np.random.randn(Din, Dout)
    x = np.tanh(x.dot(W))
    hs.append(x)
```

Forward pass for a 6-layer net with hidden size 4096

All activations tend to zero for deeper network layers

Q: What do the gradients dL/dW look like?



Weight Initialization: Activation statistics

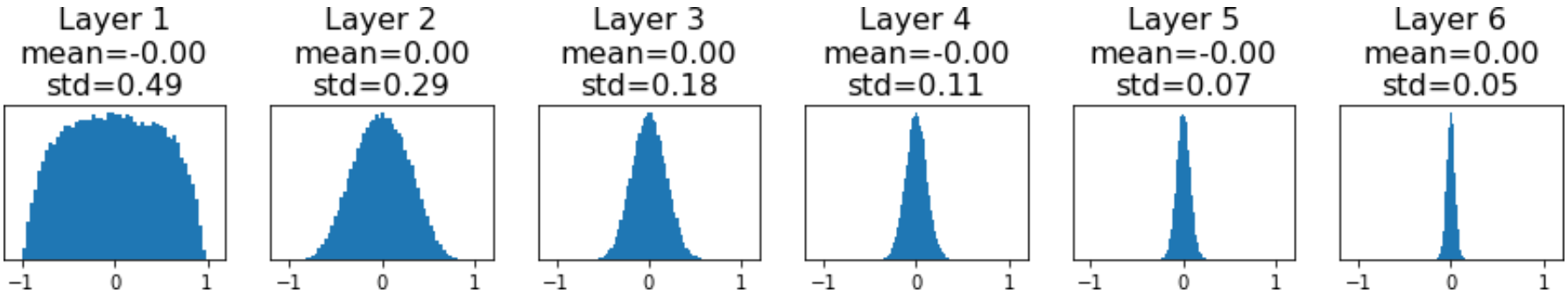
```
dims = [4096] * 7
hs = []
x = np.random.randn(16, dims[0])
for Din, Dout in zip(dims[:-1], dims[1:]):
    W = 0.01 * np.random.randn(Din, Dout)
    x = np.tanh(x.dot(W))
    hs.append(x)
```

Forward pass for a 6-layer net with hidden size 4096

All activations tend to zero for deeper network layers

Q: What do the gradients dL/dW look like?

A: All zero, no learning



Weight Initialization: Activation statistics

```
dims = [4096] * 7
hs = []
x = np.random.randn(16, dims[0])
for Din, Dout in zip(dims[:-1], dims[1:]):
    W = 0.05 * np.random.randn(Din, Dout)
    x = np.tanh(x.dot(W))
    hs.append(x)
```

Increase std of initial weights from 0.01 to 0.05

What will happen to the activations for the last layer?

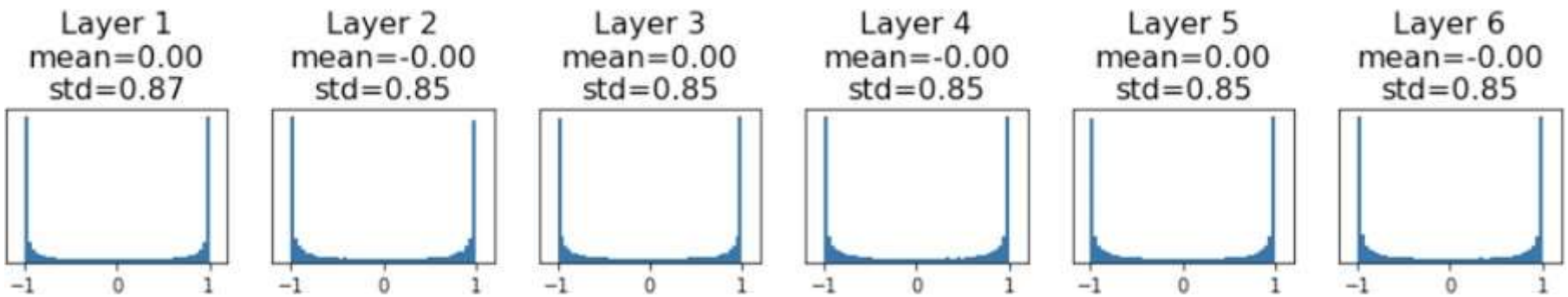
Weight Initialization: Activation statistics

```
dims = [4096] * 7
hs = []
x = np.random.randn(16, dims[0])
for Din, Dout in zip(dims[:-1], dims[1:]):
    W = 0.05 * np.random.randn(Din, Dout)
    x = np.tanh(x.dot(W))
    hs.append(x)
```

Increase std of initial weights from 0.01 to 0.05

All activations saturate

Q: What do the gradients look like?



Weight Initialization: Activation statistics

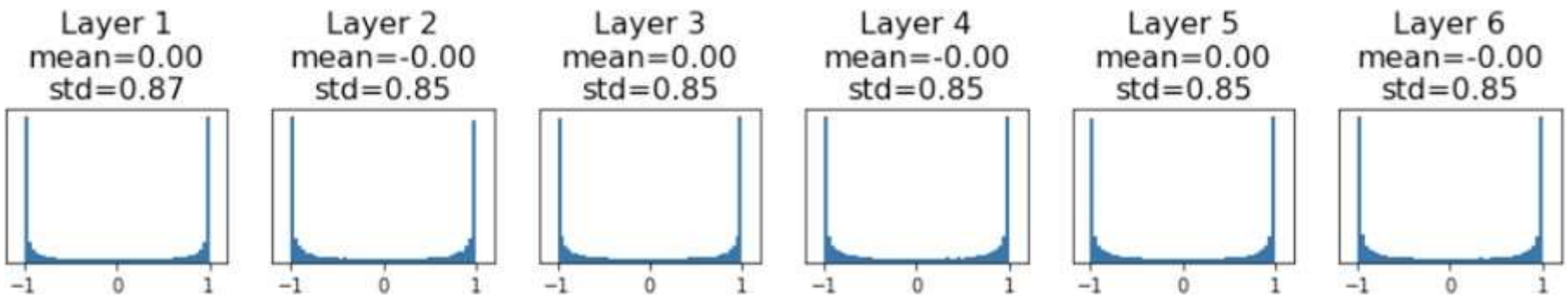
```
dims = [4096] * 7
hs = []
x = np.random.randn(16, dims[0])
for Din, Dout in zip(dims[:-1], dims[1:]):
    W = 0.05 * np.random.randn(Din, Dout)
    x = np.tanh(x.dot(W))
    hs.append(x)
```

Increase std of initial weights from 0.01 to 0.05

All activations saturate

Q: What do the gradients look like?

A: Local gradients all zero, no learning



Weight Initialization: “Xavier” Initialization

```
dims = [4096] * 7
hs = []
x = np.random.randn(16, dims[0])
for Din, Dout in zip(dims[:-1], dims[1:]):
    W = np.random.randn(Din, Dout) / np.sqrt(Din)
    x = np.tanh(x.dot(W))
    hs.append(x)
```

“Xavier” initialization:
std = 1/sqrt(Din)

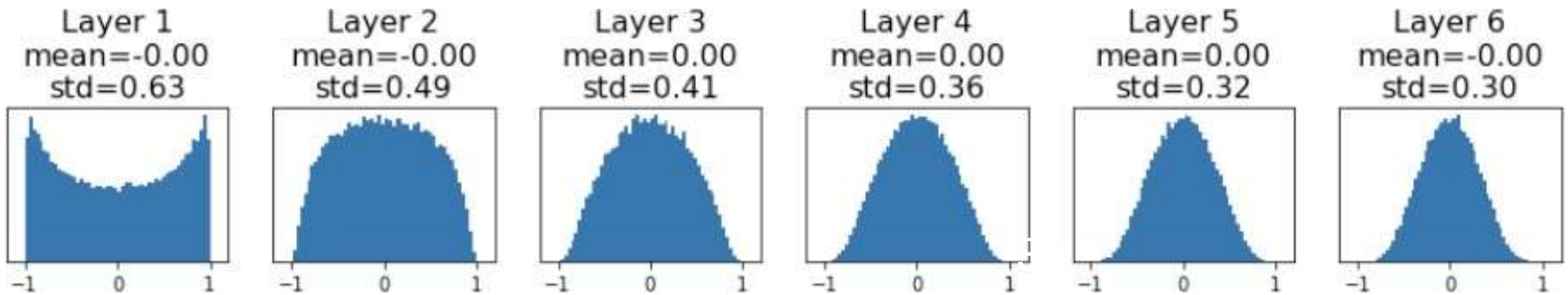
Glorot and Bengio, “Understanding the difficulty of training deep feedforward neural networks”, AISTAT 2010

Weight Initialization: “Xavier” Initialization

```
dims = [4096] * 7
hs = []
x = np.random.randn(16, dims[0])
for Din, Dout in zip(dims[:-1], dims[1:]):
    W = np.random.randn(Din, Dout) / np.sqrt(Din)
    x = np.tanh(x.dot(W))
    hs.append(x)
```

“Xavier” initialization:
std = 1/sqrt(Din)

“Just right”: Activations are nicely scaled for all layers!



Glorot and Bengio, “Understanding the difficulty of training deep feedforward neural networks”, AISTAT 2010

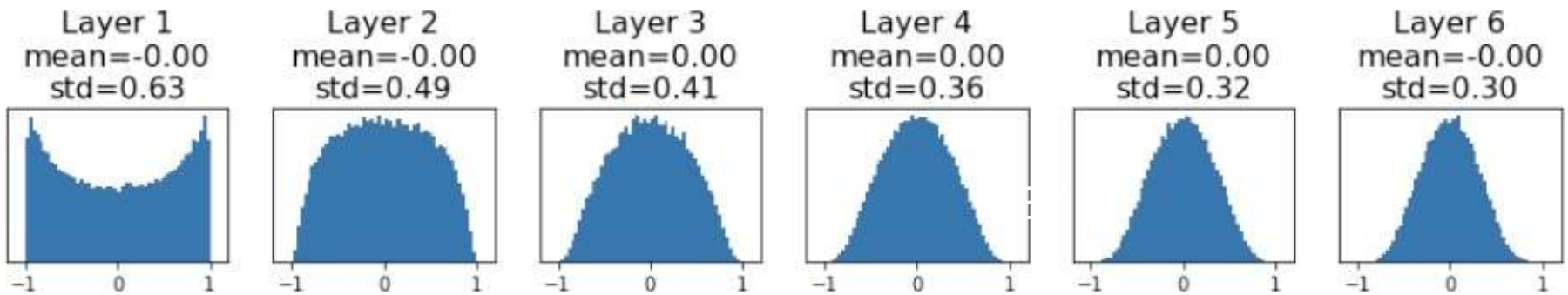
Weight Initialization: “Xavier” Initialization

```
dims = [4096] * 7
hs = []
x = np.random.randn(16, dims[0])
for Din, Dout in zip(dims[:-1], dims[1:]):
    W = np.random.randn(Din, Dout) / np.sqrt(Din)
    x = np.tanh(x.dot(W))
    hs.append(x)
```

“Xavier” initialization:
std = 1/sqrt(Din)

“Just right”: Activations are nicely scaled for all layers!

For conv layers, Din is filter_size^2 * input_channels



Glorot and Bengio, “Understanding the difficulty of training deep feedforward neural networks”, AISTAT 2010

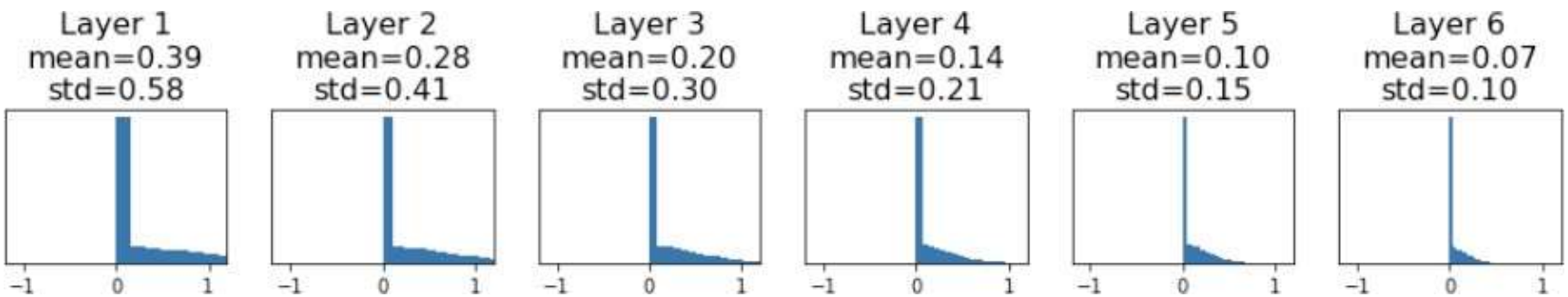
Weight Initialization: What about ReLU?

```
dims = [4096] * 7
hs = []
x = np.random.randn(16, dims[0])
for Din, Dout in zip(dims[:-1], dims[1:]):
    W = np.random.randn(Din, Dout) / np.sqrt(Din)
    x = np.maximum(0, x.dot(W))
    hs.append(x)
```

Change from tanh to ReLU

Xavier assumes zero centered activation function

Activations collapse to zero again, no learning

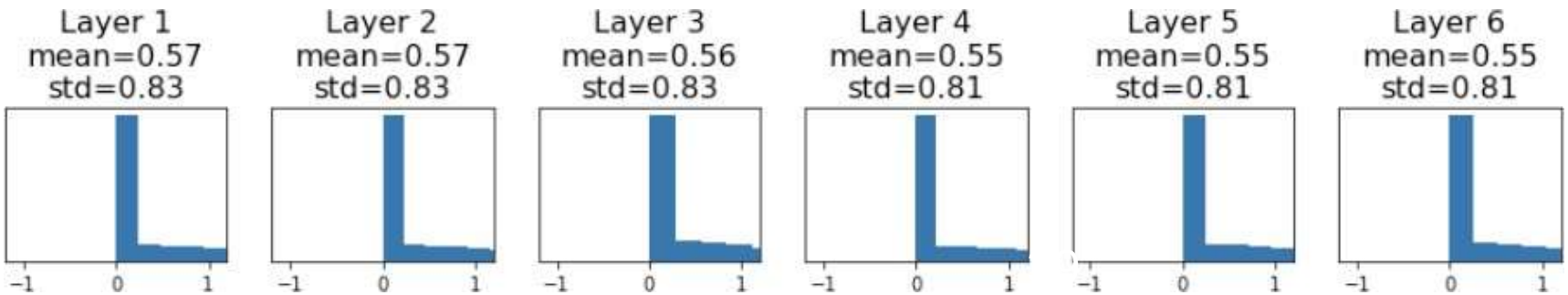


Weight Initialization: Kaiming / MSRA Initialization

```
dims = [4096] * 7
hs = []
x = np.random.randn(16, dims[0])
for Din, Dout in zip(dims[:-1], dims[1:]):
    W = np.random.randn(Din, Dout) * np.sqrt(2/Din)
    x = np.maximum(0, x.dot(W))
    hs.append(x)
```

ReLU correction: std = sqrt(2 / Din)

"Just right": Activations are nicely scaled for all layers!



He et al, "Delving Deep into Rectifiers: Surpassing Human-Level Performance on ImageNet Classification", ICCV 2015

Code: Weight Initialization in PyTorch

PYTHON / PYTORCH

```python
import torch.nn as nn
import torch.nn.init as init

class ConvNet(nn.Module):
    def __init__(self):
        super().__init__()
        self.conv1 = nn.Conv2d(3, 64, 3, padding=1)
        self.conv2 = nn.Conv2d(64, 128, 3, padding=1)
        self.fc = nn.Linear(128 * 8 * 8, 10)
        self._init_weights()

    def _init_weights(self):
        for m in self.modules():
            if isinstance(m, nn.Conv2d):
                # Kaiming init for ReLU networks
                init.kaiming_normal_(m.weight, mode='fan_out',
                                     nonlinearity='relu')
            elif isinstance(m, nn.Linear):
                # Xavier init for linear layers
                init.xavier_normal_(m.weight)
                init.zeros_(m.bias)
```

When to Use What

ReLU / LeakyReLU layers:

Kaiming (He) initialization

`std = sqrt(2 / fan_in)`

Sigmoid / tanh layers:

Xavier (Glorot) initialization

`std = sqrt(1 / fan_in)`

Transformers / LayerNorm:

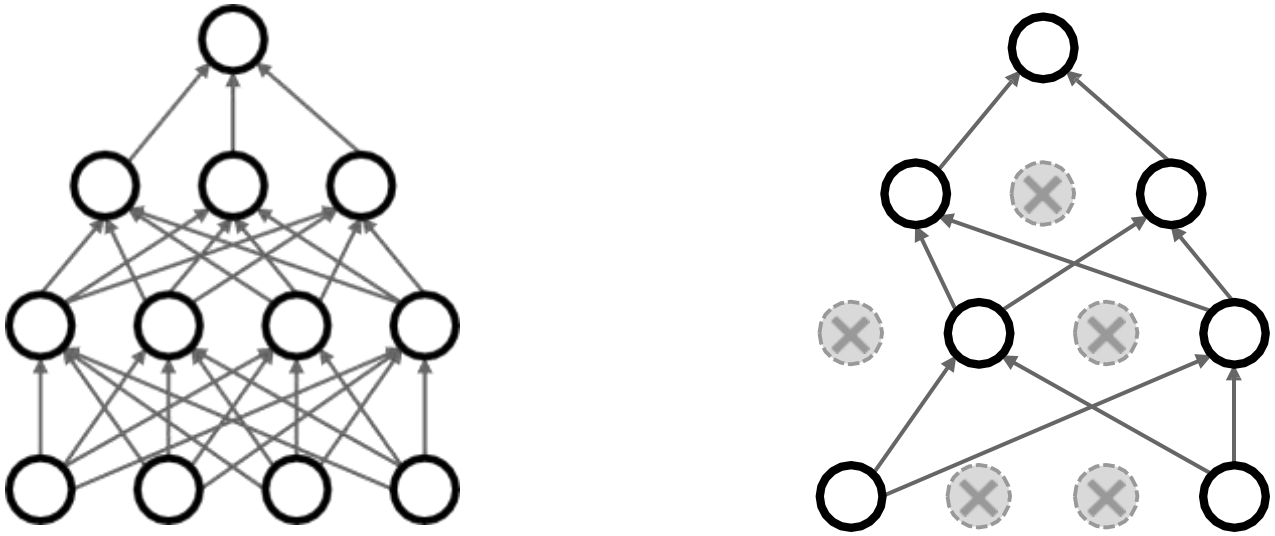
Usually handled by the framework (e.g. nn.Transformer). Use default init.

Never use constant init (all same value) — breaks symmetry requirement.

Regularization

Regularization: Dropout

In each forward pass, randomly set some neurons to zero
Probability of dropping is a hyperparameter; 0.5 is common

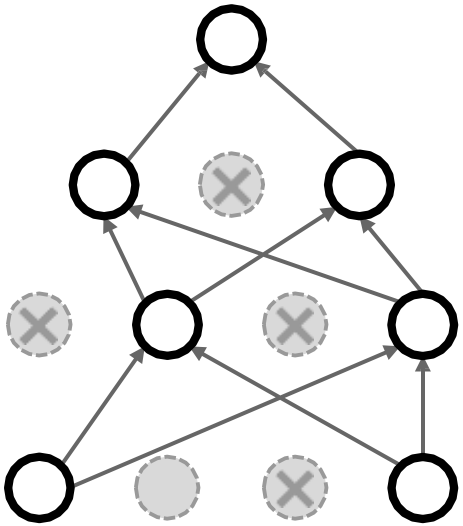


Srivastava et al, “Dropout: A simple way to prevent neural networks from overfitting”, JMLR 2014

Regularization: Dropout

```python
p = 0.5 # probability of keeping a unit active. higher = less dropout

def train_step(X):
  """ X contains the data """

  # forward pass for example 3-layer neural network
  H1 = np.maximum(0, np.dot(W1, X) + b1)
  U1 = np.random.rand(*H1.shape) < p # first dropout mask
  H1 *= U1 # drop!
  H2 = np.maximum(0, np.dot(W2, H1) + b2)
  U2 = np.random.rand(*H2.shape) < p # second dropout mask
  H2 *= U2 # drop!
  out = np.dot(W3, H2) + b3

  # backward pass: compute gradients... (not shown)
  # perform parameter update... (not shown)
```

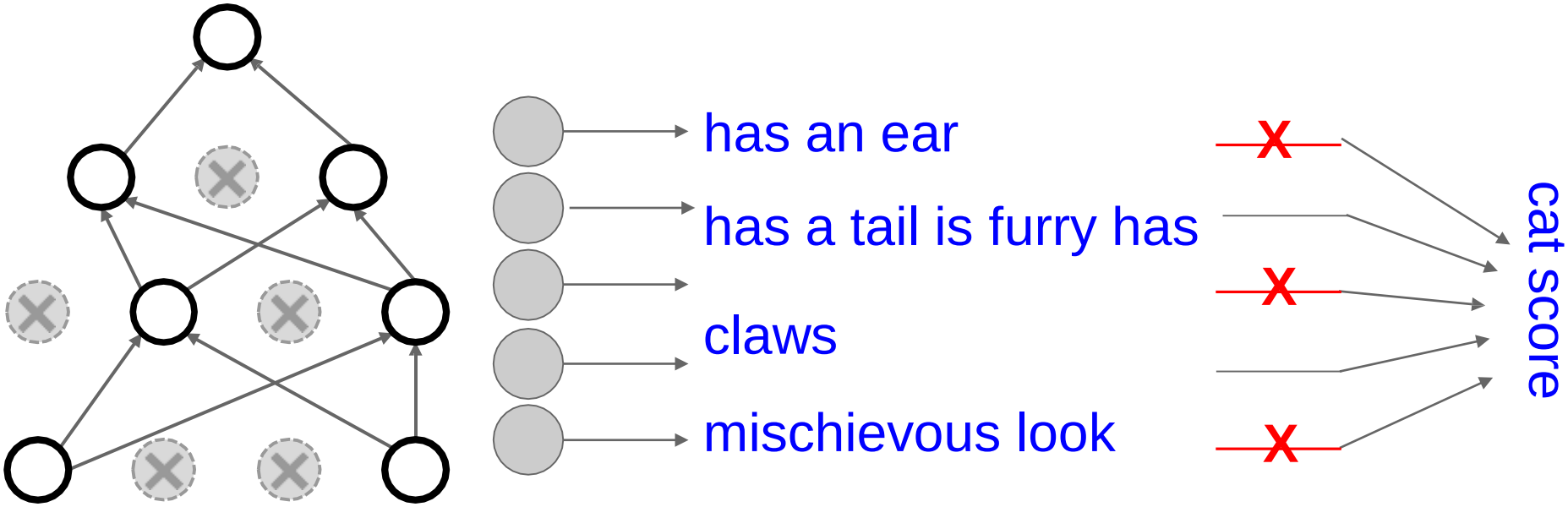
Example forward pass with a 3- layer network using dropout



Regularization: Dropout

How can this possibly be a good idea?

Forces the network to have a redundant representation



Regularization: Dropout

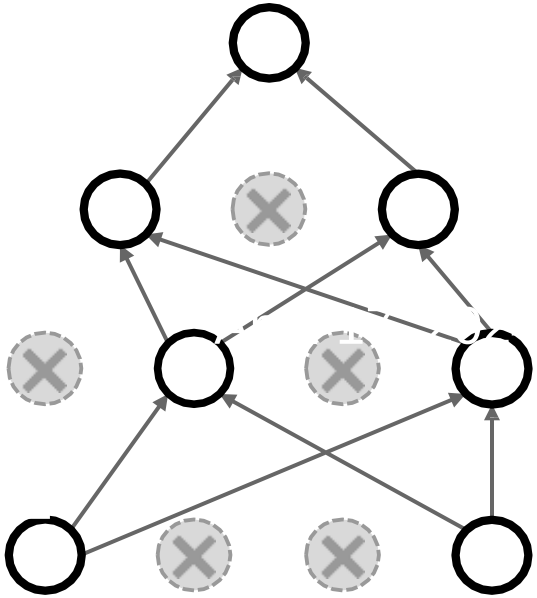
How can this possibly be a good idea?

Another interpretation:

Dropout is training a large **ensemble** of models (that share parameters).

Each binary mask is one model

An FC layer with 4096 units has
$2^{4096} \sim 10^{1233}$ possible masks!
Only $\sim 10^{82}$ atoms in the universe...



Dropout: Test time

Dropout makes our output random!

Output (label) Input (image) Random mask

$$\boxed{y} = f_W(\boxed{x}, \boxed{z})$$

Want to "average out" the randomness at test-time

$$y = f(x) = E_z\left[f(x,z)\right] = \int p(z) f(x,z) dz$$

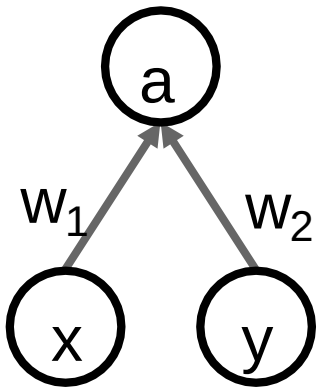
But this integral seems hard …

Dropout: Test time

Want to approximate the integral

Consider a single neuron.

$$y = f(x) = E_z\big[f(x,z)\big] = \int p(z) f(x,z) dz$$

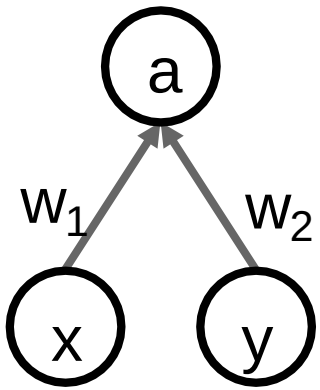


Dropout: Test time

Want to approximate the integral

$$y = f(x) = E_z\big[f(x,z)\big] = \int p(z) f(x,z) dz$$

Consider a single neuron.



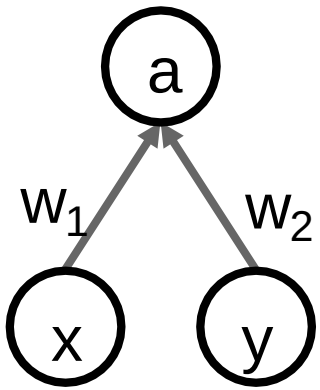
At test time we have: $E\big[a\big] = w_1 x + w_2 y$

Dropout: Test time

Want to approximate the integral

Consider a single neuron.

$$y = f(x) = E_z\big[f(x,z)\big] = \int p(z) f(x,z) dz$$



At test time we have: $E\big[a\big] = w_1 x + w_2 y$

During training we have:

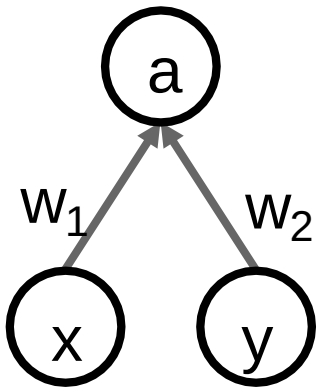
$$\begin{aligned} E\big[a\big] =& \frac{1}{4}(w_1 x + w_2 y) + \frac{1}{4}(w_1 x + 0y) \\ &+ \frac{1}{4}(0x + 0y) + \frac{1}{4}(0x + w_2 y) \\ =& \frac{1}{2}(w_1 x + w_2 y) \end{aligned}$$

Dropout: Test time

Want to approximate the integral

$$y = f(x) = E_z\left[f(x,z)\right] = \int p(z) f(x,z) dz$$

Consider a single neuron.



At test time we have: $E\left[a\right] = w_1 x + w_2 y$

During training we have:

$$\begin{aligned} E\left[a\right] =& \frac{1}{4}(w_1 x + w_2 y) + \frac{1}{4}(w_1 x + 0y) \\ &+ \frac{1}{4}(0x + 0y) + \frac{1}{4}(0x + w_2 y) \\ =& \frac{1}{2}(w_1 x + w_2 y) \end{aligned}$$

At test time, **multiply** by dropout probability

Dropout: Test time

```
def predict(X):
  # ensembled forward pass
  H1 = np.maximum(0, np.dot(W1, X) + b1) * p # NOTE: scale the activations
  H2 = np.maximum(0, np.dot(W2, H1) + b2) * p # NOTE: scale the activations
  out = np.dot(W3, H2) + b3
```

At test time all neurons are active always
=> We must scale the activations so that for each neuron:
<u>output at test time</u> = <u>expected output at training time</u>

Dropout Summary

```
""" Vanilla Dropout: Not recommended implementation (see notes below) """

p = 0.5 # probability of keeping a unit active. higher = less dropout

def train_step(X):
  """ X contains the data """

  # forward pass for example 3-layer neural network
  H1 = np.maximum(0, np.dot(W1, X) + b1)
  U1 = np.random.rand(*H1.shape) < p # first dropout mask
  H1 *= U1 # drop!
  H2 = np.maximum(0, np.dot(W2, H1) + b2)
  U2 = np.random.rand(*H2.shape) < p # second dropout mask
  H2 *= U2 # drop!
  out = np.dot(W3, H2) + b3

  # backward pass: compute gradients... (not shown)
  # perform parameter update... (not shown)

def predict(X):
  # ensembled forward pass
  H1 = np.maximum(0, np.dot(W1, X) + b1) * p # NOTE: scale the activations
  H2 = np.maximum(0, np.dot(W2, H1) + b2) * p # NOTE: scale the activations
  out = np.dot(W3, H2) + b3
```

drop in train time

scale at test time

More common: “Inverted dropout”

```
p = 0.5 # probability of keeping a unit active. higher = less dropout

def train_step(X):
  # forward pass for example 3-layer neural network
  H1 = np.maximum(0, np.dot(W1, X) + b1)
  U1 = (np.random.rand(*H1.shape) < p) / p # first dropout mask. Notice /p!
  H1 *= U1 # drop!
  H2 = np.maximum(0, np.dot(W2, H1) + b2)
  U2 = (np.random.rand(*H2.shape) < p) / p # second dropout mask. Notice /p!
  H2 *= U2 # drop!
  out = np.dot(W3, H2) + b3

  # backward pass: compute gradients... (not shown)
  # perform parameter update... (not shown)

def predict(X):
  # ensembled forward pass
  H1 = np.maximum(0, np.dot(W1, X) + b1) # no scaling necessary
  H2 = np.maximum(0, np.dot(W2, H1) + b2)
  out = np.dot(W3, H2) + b3
```

test time is unchanged!

Code: Dropout & Regularization

PYTHON / PYTORCH

```
import torch.nn as nn

class RegularizedNet(nn.Module):
    def __init__(self):
        super().__init__()
        self.features = nn.Sequential(
            nn.Linear(784, 512),
            nn.ReLU(),
            nn.Dropout(p=0.5),       # Drop 50% during training
            nn.Linear(512, 256),
            nn.ReLU(),
            nn.Dropout(p=0.3),       # Less dropout in later
layers
            nn.Linear(256, 10)
        )
    def forward(self, x):
        return self.features(x)

# IMPORTANT: toggle train/eval mode
model.train()   # Enables dropout (training)
model.eval()    # Disables dropout (inference)

# Weight decay (L2 regularization) in optimizer
optimizer = torch.optim.Adam(model.parameters(),
                             lr=1e-3, weight_decay=1e-4)
```

Regularization Checklist

Dropout rates:

FC layers: p = 0.5 (typical)

Conv layers: p = 0.1–0.3

After attention: p = 0.1

Weight decay:

Start with 1e-4, try 1e-5 or 0

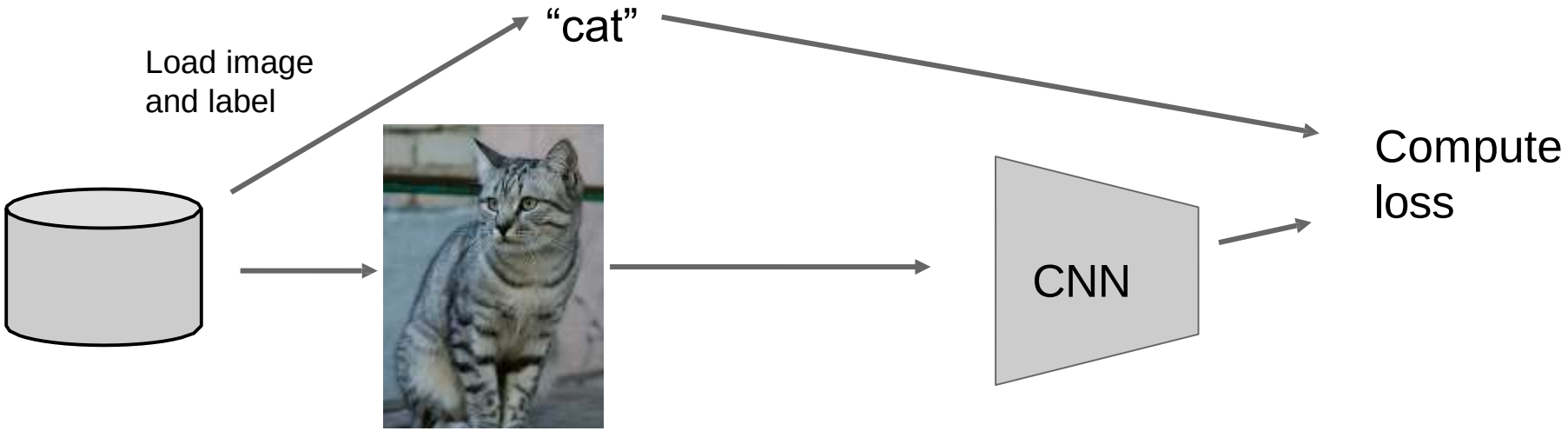
Common mistake:

Forgetting model.eval() at test time! This keeps dropout on and produces noisy predictions.

Order of operations:

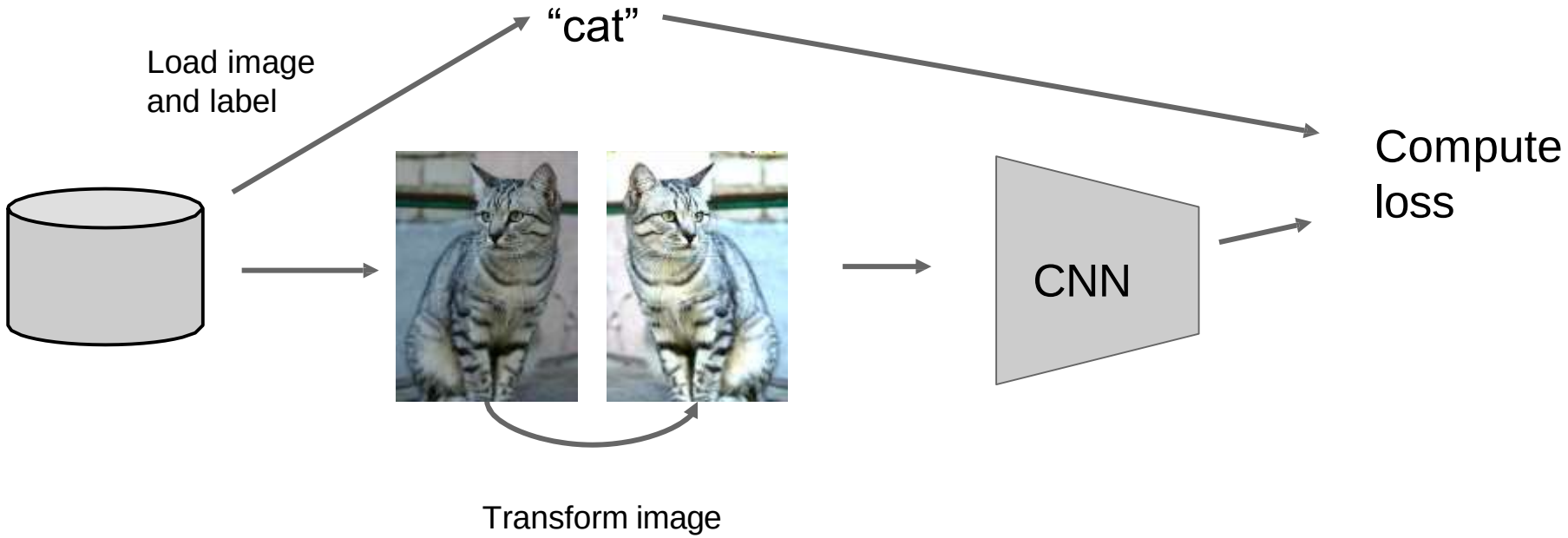
`Linear → BatchNorm → Activation → Dropout`

Regularization: Data Augmentation



This image by Nikita is
licensed under CC-BY 2.0

Regularization: Data Augmentation



Data Augmentation
Horizontal Flips

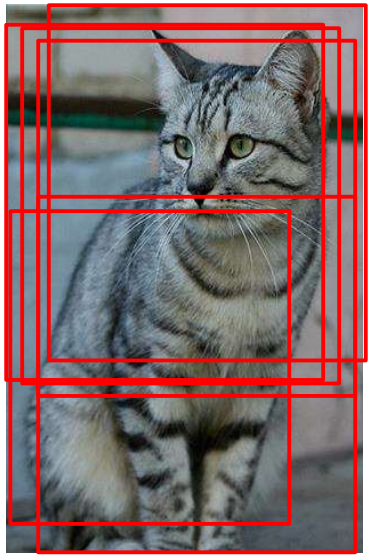
Data Augmentation

Random crops and scales

Training: sample random crops / scales

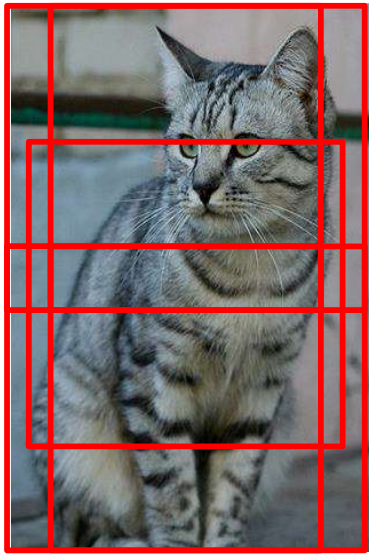
ResNet:

1. Pick random L in range [256, 480]
2. Resize training image, short side = L
3. Sample random 224 x 224 patch



Data Augmentation

Random crops and scales



Training: sample random crops / scales

ResNet:

1. Pick random L in range [256, 480]
2. Resize training image, short side = L
3. Sample random 224 x 224 patch

Testing: average a fixed set of crops

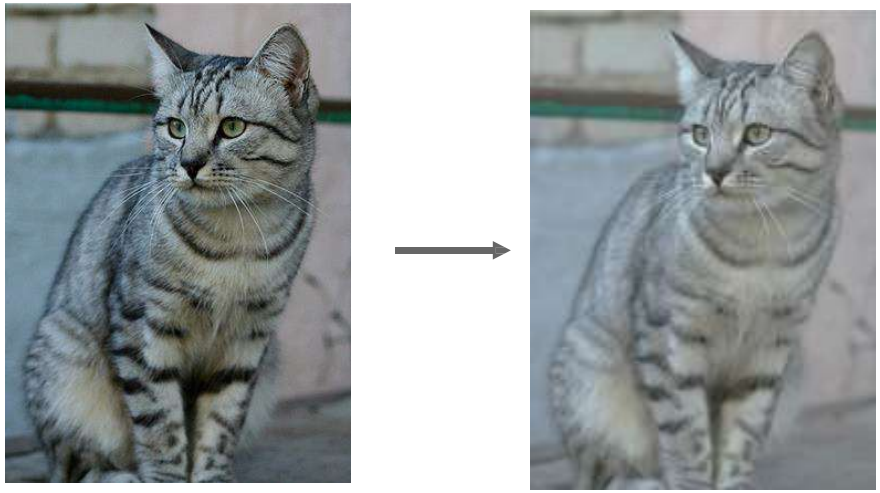
ResNet:

1. Resize image at 5 scales: {224, 256, 384, 480, 640}
2. For each size, use 10 224 x 224 crops: 4 corners + center, + flips

Data Augmentation

Color Jitter

Simple: Randomize contrast and brightness



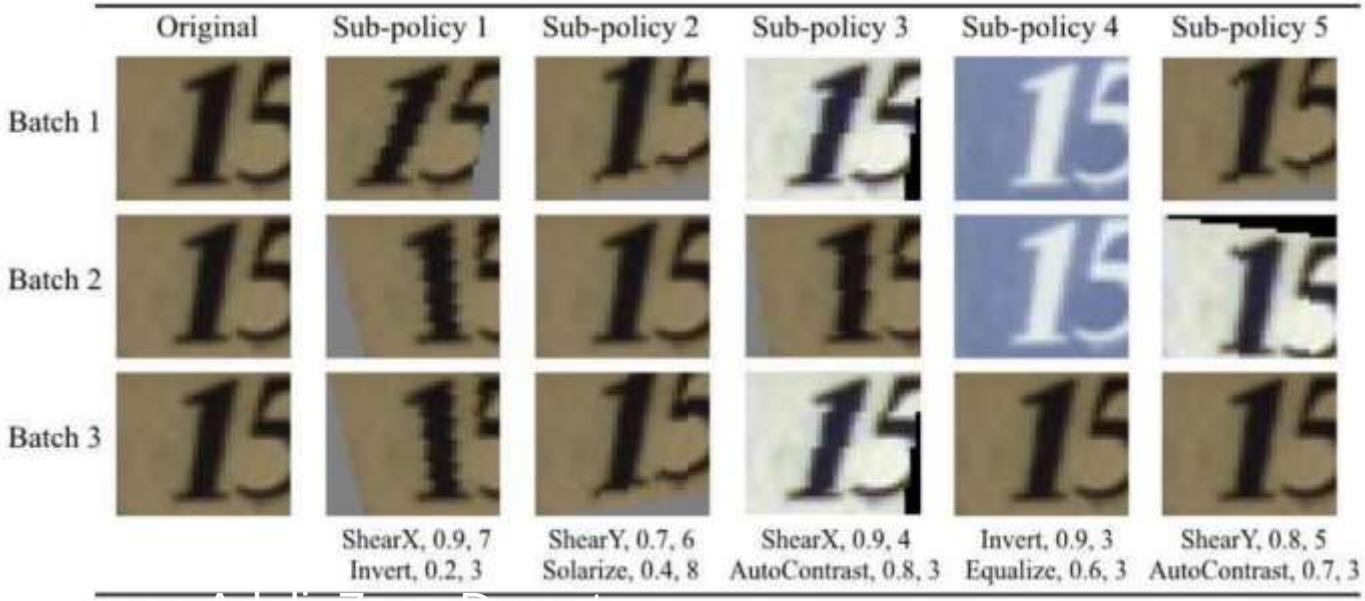
Data Augmentation

Get creative for your problem!

Examples of data augmentations:

- translation
- rotation
- stretching
- shearing,
- lens distortions, …

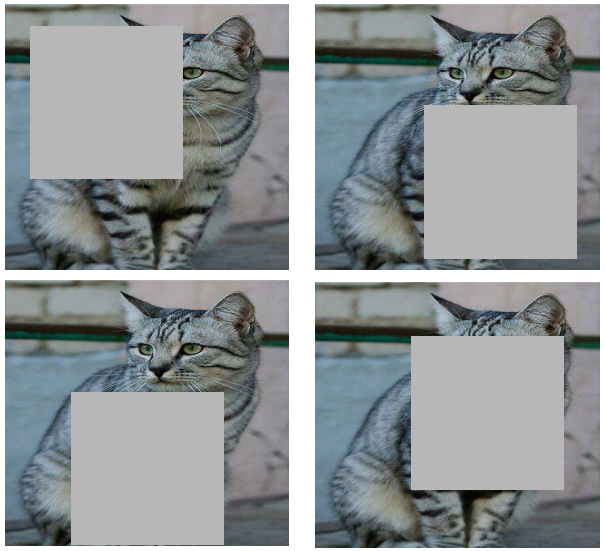
Automatic Data Augmentation



Cubuk et al., “AutoAugment: Learning Augmentation Strategies from Data”, CVPR 2019

Regularization: Cutout

Training: Set random image regions to zero
Testing: Use full image



DeVries and Taylor, “Improved Regularization of Convolutional Neural Networks with Cutout”, arXiv 2017

Code: Data Augmentation Pipeline

PYTHON / TORCHVISION

```
from torchvision import transforms

# Training transforms (apply augmentation)
train_transform = transforms.Compose([
    transforms.RandomResizedCrop(224, scale=(0.8, 1.0)),
    transforms.RandomHorizontalFlip(),
    transforms.ColorJitter(brightness=0.2, contrast=0.2,
                           saturation=0.2, hue=0.1),
    transforms.RandomErasing(p=0.25),  # Cutout-like
    transforms.ToTensor(),
    transforms.Normalize(mean=[0.485, 0.456, 0.406],
                         std=[0.229, 0.224, 0.225]),
])

# Validation transforms (NO augmentation)
val_transform = transforms.Compose([
    transforms.Resize(256),
    transforms.CenterCrop(224),
    transforms.ToTensor(),
    transforms.Normalize(mean=[0.485, 0.456, 0.406],
                         std=[0.229, 0.224, 0.225]),
])
```

Key Points

Always augment training only

Validation/test data should use deterministic transforms.

Start simple:

Flip + Crop + Normalize is a strong baseline.

Add progressively:

ColorJitter, RandomErasing, or AutoAugment if underfitting is not a problem.

Domain matters:

Medical images: don't flip vertically if orientation matters. Satellite: rotation is fine. Text: no horizontal flip.

Optimization toolkit: mini-batch SGD

- We optimize parameters θ to minimize loss $L(\theta)$ using gradients $\nabla_\theta L$
- Mini-batch SGD baseline: compute gradient on a batch B and update
  - $\theta \leftarrow \theta - \eta \cdot (1/|B|) \sum_{i \in B} \nabla_\theta L_i$
- Why mini-batches? Efficient (vectorized), less noisy than batch size 1, cheaper than full batch
- Key knobs: learning rate η, batch size $|B|$, weight decay / regularization

Momentum & Adam (practical defaults)

- Momentum smooths noisy gradients and accelerates along consistent directions
  - $v \leftarrow \mu v - \eta \nabla_\theta L \quad ; \quad \theta \leftarrow \theta + v \quad (\mu \approx 0.9)$
- Adam ≈ Momentum + per-parameter step sizes (RMS of gradients) (+ bias correction)
- Good starting point: Adam ($\eta \approx 1e-3$) or SGD+Momentum (η tuned, $\mu \approx 0.9$)
- If unstable: lower η first; if too slow: raise η or add momentum

Batch size: speed vs generalization

- Small batches: noisier updates → can help generalization, but slower / less stable
- Large batches: efficient hardware utilization, smoother gradients, may generalize worse
- Rule of thumb: start with 64–256 (fit to memory), then tune
- Batch size interacts with learning rate (often increase η when increasing batch size)

Learning-rate schedules (why decay helps)

- Large η helps make fast progress early; smaller η helps “settle” near a good solution
- Common schedules:
  - Step decay (drop η by 10× at milestones)
  - Cosine decay (popular default), often with short warm-up
  - Exponential / 1/t
- Recipe: get training working with constant η → then add decay for best accuracy

Training curves: diagnose → fix

- Plot training & validation loss/accuracy; keep a fixed eval subset to reduce noise
- Loss explodes / oscillates → decrease η; check init; try Adam / momentum
- Loss decreases very slowly → increase η; add momentum; check data preprocessing
- Train improves but val worsens → overfitting: regularize / augment / early stop
- Both are bad → underfitting: bigger model, train longer, reduce regularization

Choosing Hyperparameters

(without tons of GPUs)

Choosing Hyperparameters

Step 1: Check initial loss

Turn off weight decay, sanity check loss at initialization
e.g. log(C) for softmax with C classes

Random guessing ➔ 1/C probability for each class
Softmax Loss ➔ -log(1/C) = log(C)

Choosing Hyperparameters

Step 1: Check initial loss
Step 2: Overfit a small sample

Try to train to 100% training accuracy on a small sample of training data (~5-10 minibatches); fiddle with architecture, learning rate, weight initialization

Loss not going down? LR too low, bad initialization
Loss explodes to Inf or NaN? LR too high, bad initialization

Choosing Hyperparameters

Step 1: Check initial loss
Step 2: Overfit a small sample
Step 3: Find LR that makes loss go down

Use the architecture from the previous step, use all training data, turn on small weight decay, find a learning rate that makes the loss drop significantly within ~100 iterations

Good learning rates to try: 1e-1, 1e-2, 1e-3, 1e-4

Choosing Hyperparameters

Step 1: Check initial loss
Step 2: Overfit a small sample
Step 3: Find LR that makes loss go down
Step 4: Coarse grid, train for ~1-5 epochs

Choose a few values of learning rate and weight decay around what worked from Step 3, train a few models for ~1-5 epochs.

Good weight decay to try: 1e-4, 1e-5, 0

Choosing Hyperparameters

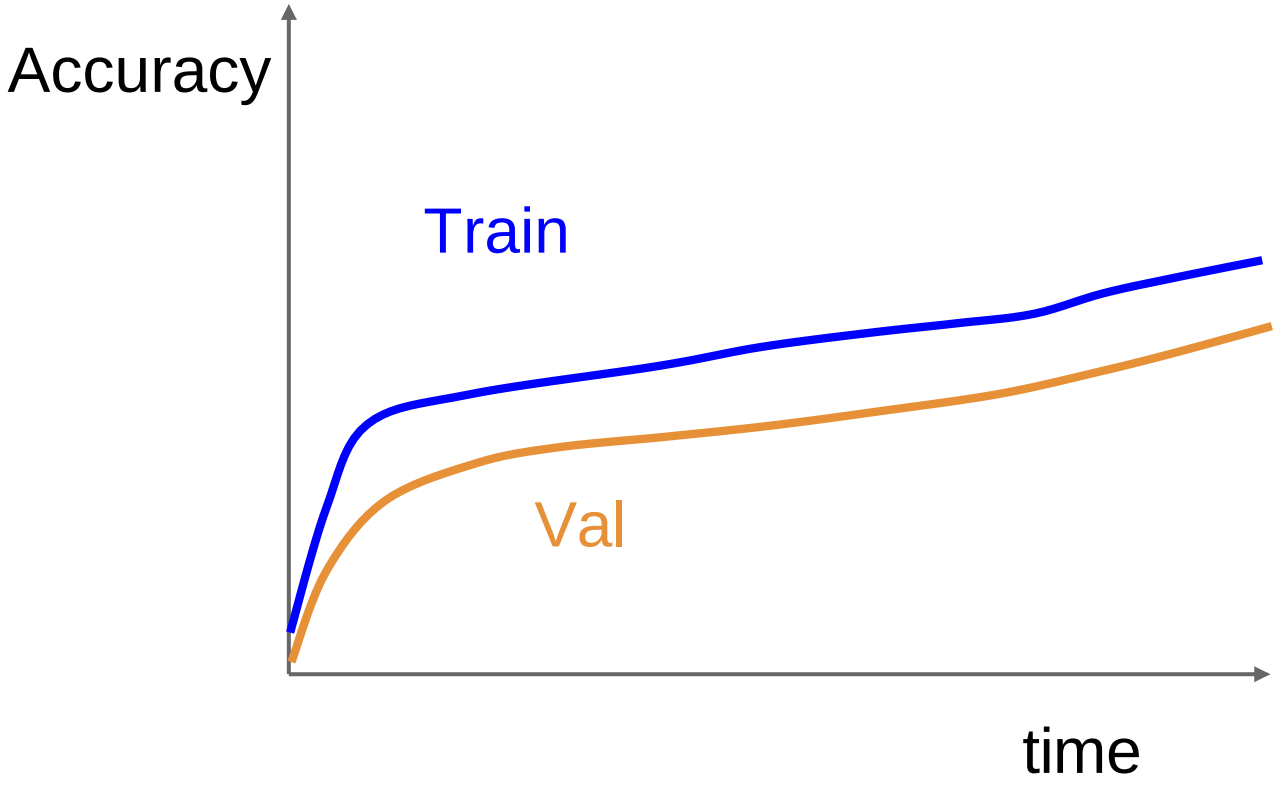
Step 1: Check initial loss
Step 2: Overfit a small sample
Step 3: Find LR that makes loss go down **Step 4**: Coarse grid, train for ~1-5 epochs **Step 5**: Refine grid, train longer

Pick best models from Step 4, train them for longer (~10-20 epochs) with constant learning rate

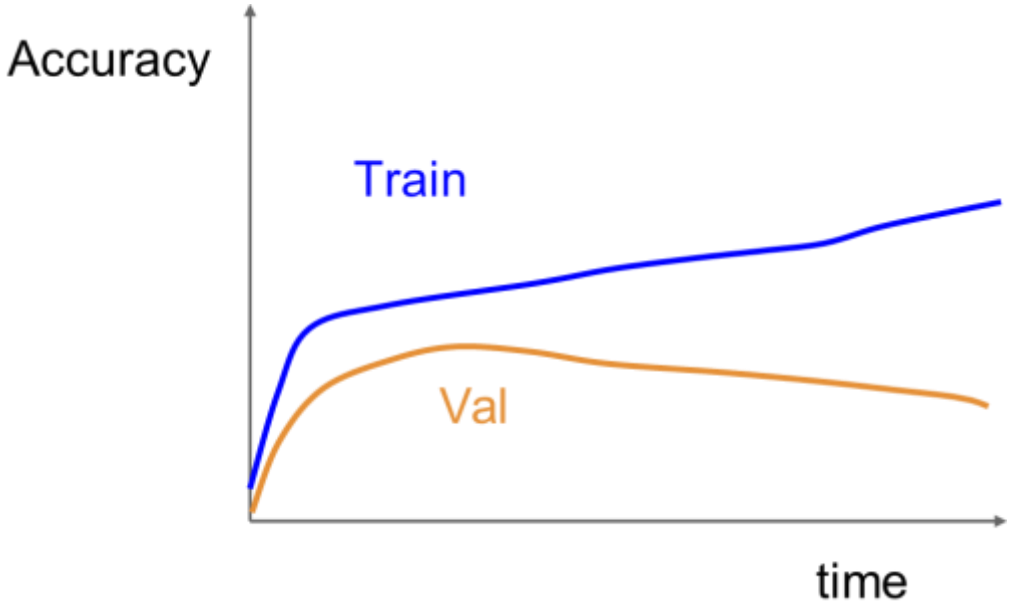
Choosing Hyperparameters

Step 1: Check initial loss
Step 2: Overfit a small sample
Step 3: Find LR that makes loss go down
Step 4: Coarse grid, train for ~1-5 epochs
Step 5: Refine grid, train longer
Step 6: Look at loss and accuracy curves

Accuracy still going up, you need to train longer

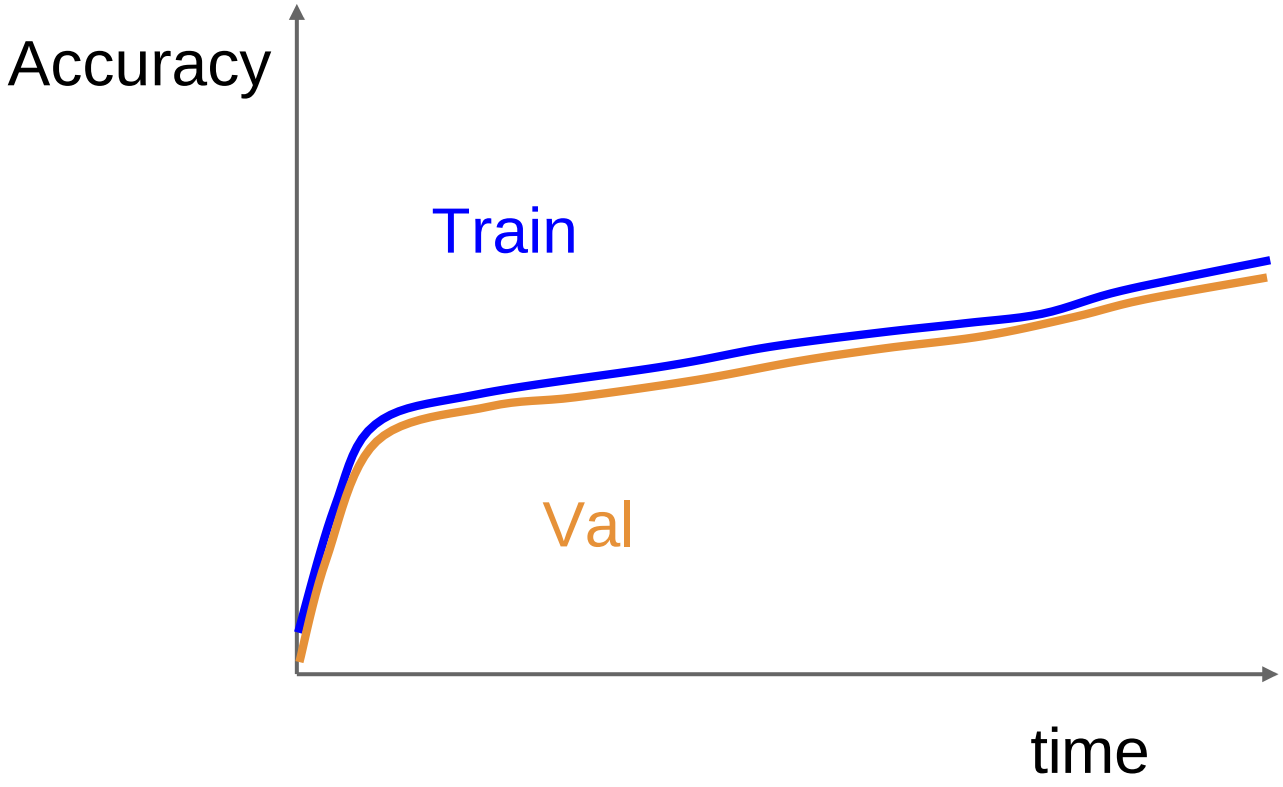


Huge train / val gap



- Regularization (Dropout, BatchNorm, Weight Decay) is Only Applied During Training
- Training and Validation data sets are not from the same distribution
- Error in the implementation

No gap between train / val means underfitting: train longer, can use a bigger model



Hands-On: Plotting & Reading Loss Curves

DIAGNOSTIC CODE

```
import matplotlib.pyplot as plt

def plot_training_curves(train_losses, val_losses,
                         train_accs, val_accs):
    fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(12, 4))

    ax1.plot(train_losses, label='Train Loss')
    ax1.plot(val_losses, label='Val Loss')
    ax1.set_xlabel('Epoch'); ax1.set_ylabel('Loss')
    ax1.legend(); ax1.set_title('Loss Curves')

    ax2.plot(train_accs, label='Train Acc')
    ax2.plot(val_accs, label='Val Acc')
    ax2.set_xlabel('Epoch'); ax2.set_ylabel('Accuracy')
    ax2.legend(); ax2.set_title('Accuracy Curves')

    plt.tight_layout(); plt.savefig('curves.png')
    plt.show()
```

TENSORBOARD

```
# Alternative: Use TensorBoard for real-time monitoring
from torch.utils.tensorboard import SummaryWriter
writer = SummaryWriter('runs/experiment_1')
writer.add_scalar('Loss/train', loss.item(), epoch)
writer.add_scalar('Loss/val', val_loss, epoch)  # then run:
tensorboard --logdir=runs
```

What to Look For

✓ Loss decreasing smoothly

Training is healthy.

⚠ Train loss ↓ but val loss ↑

Overfitting! Add regularization, dropout, data augmentation, or reduce model size.

⚠ Both losses plateau early

Underfitting. Increase model capacity, train longer, or raise learning rate.

✗ Loss explodes / NaN

LR too high or bad init. Reduce LR by 10x, check initialization.

✗ Loss stays flat from start

LR too low or bug. Increase LR, verify data loading and labels.

Transfer learning

Hands-On: Fixing Common Training Issues

OVERFITTING (large train/val gap)

```
# 1. Add dropout
self.dropout = nn.Dropout(0.5)

# 2. Add weight decay
optimizer = optim.Adam(params, lr=1e-3,
                       weight_decay=1e-4)

# 3. Add data augmentation
transforms.RandomHorizontalFlip()
transforms.RandomResizedCrop(224)

# 4. Reduce model size
# Use fewer layers or hidden units

# 5. Early stopping
if val_loss > best_val_loss:
    patience_counter += 1
if patience_counter >= patience:
    break  # stop training
```

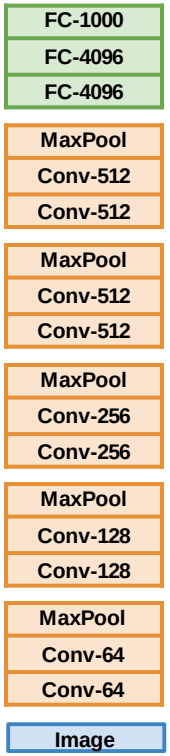
UNDERFITTING (low train & val accuracy)

```
# 1. Train longer
num_epochs = 100  # increase from e.g. 20

# 2. Increase learning rate
# Try 1e-3 instead of 1e-4
scheduler = optim.lr_scheduler.StepLR(
    optimizer, step_size=30, gamma=0.1)

# 3. Bigger model
nn.Linear(512, 1024)  # more hidden units
# Add more layers

# 4. Reduce regularization
# Lower dropout: 0.5 -> 0.2 or remove
# Lower weight_decay: 1e-4 -> 1e-5

# 5. Check for bugs
# Verify: labels correct? data loading ok?
# Sanity check: overfit 1 batch first
```

Transfer Learning with CNNs

Donahue et al, “DeCAF: A Deep Convolutional Activation Feature for Generic Visual Recognition”, ICML 2014 Razavian et al, “CNN Features Off-the-Shelf: An Astounding Baseline for Recognition”, CVPR Workshops 2014

1. Train on Imagenet



Putting It All Together: Training Loop

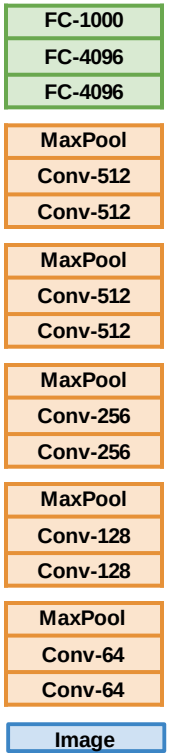
COMPLETE EXAMPLE

```
import torch
import torch.nn as nn
import torch.optim as optim

# --- Setup ---
model = ConvNet()                                # Use proper weight init (Kaiming)
optimizer = optim.Adam(model.parameters(), lr=1e-3, weight_decay=1e-4)  # L2 reg
scheduler = optim.lr_scheduler.CosineAnnealingLR(optimizer, T_max=num_epochs)
criterion = nn.CrossEntropyLoss()

# --- Training loop ---
best_val_acc = 0
for epoch in range(num_epochs):
    model.train()                                # Enable dropout / batchnorm train mode
    for images, labels in train_loader:
        outputs = model(images)
        loss = criterion(outputs, labels)
        optimizer.zero_grad()
        loss.backward()
        optimizer.step()

    # --- Validation ---
    model.eval()                                 # Disable dropout for evaluation
    with torch.no_grad():
        val_loss, correct, total = 0, 0, 0
        for images, labels in val_loader:
            outputs = model(images)
            val_loss += criterion(outputs, labels).item()
            correct += (outputs.argmax(1) == labels).sum().item()
            total += labels.size(0)
    val_acc = correct / total
    scheduler.step()
    print(f"Epoch {epoch}: train_loss={loss:.4f}, val_acc={val_acc:.4f}, lr={scheduler.get_last_lr()[0]:.6f}")

    if val_acc > best_val_acc:                   # Save best model
        best_val_acc = val_acc
        torch.save(model.state_dict(), 'best_model.pth')
```

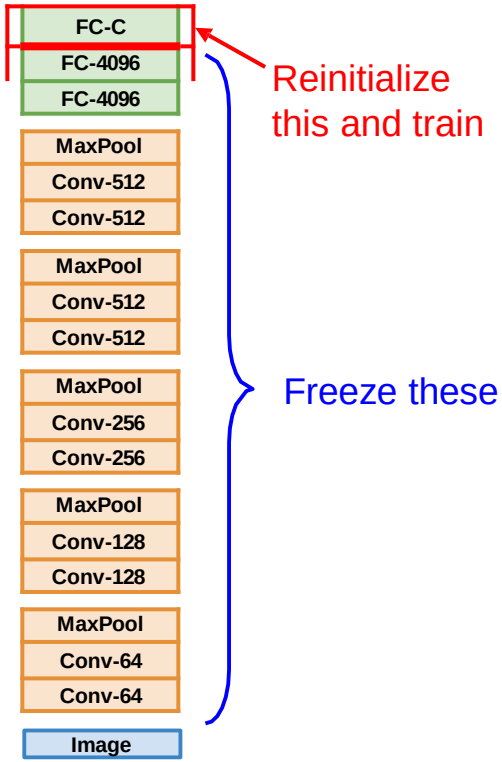
Transfer Learning with CNNs

Donahue et al, "DeCAF: A Deep Convolutional Activation Feature for Generic Visual Recognition", ICML 2014
Razavian et al, "CNN Features Off-the-Shelf: An Astounding Baseline for Recognition", CVPR Workshops 2014

1. Train on Imagenet



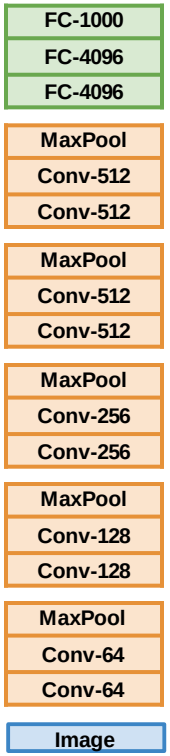
2. Small Dataset (C classes)



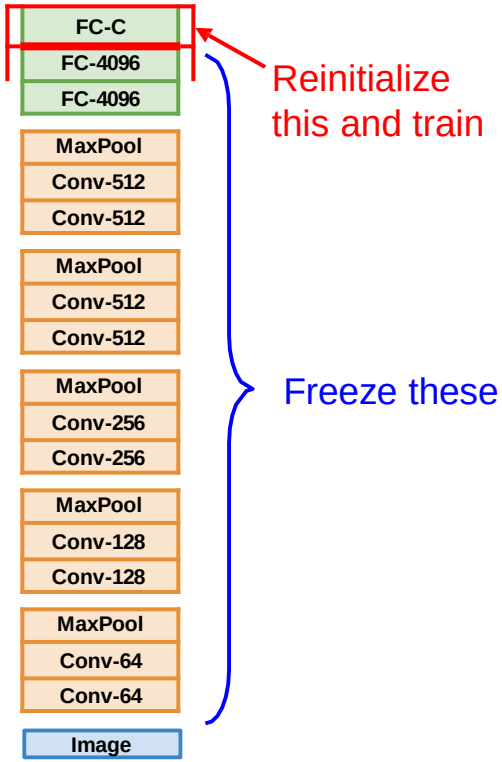
Transfer Learning with CNNs

Donahue et al, "DeCAF: A Deep Convolutional Activation Feature for Generic Visual Recognition", ICML 2014
Razavian et al, "CNN Features Off-the-Shelf: An Astounding Baseline for Recognition", CVPR Workshops 2014

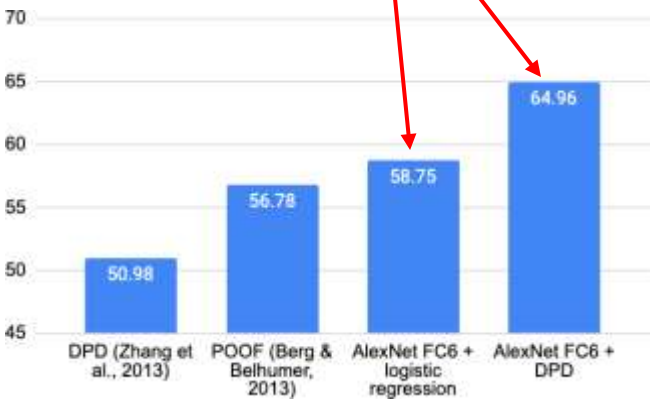
1. Train on Imagenet



2. Small Dataset (C classes)



Finetuned from AlexNet

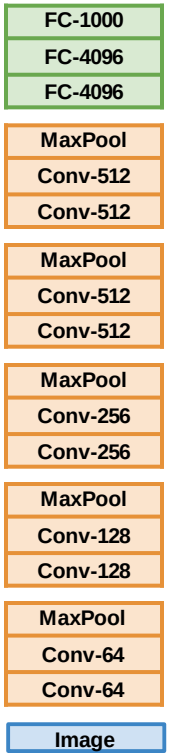


Donahue et al, "DeCAF: A Deep Convolutional Activation Feature for Generic Visual Recognition", ICML 2014

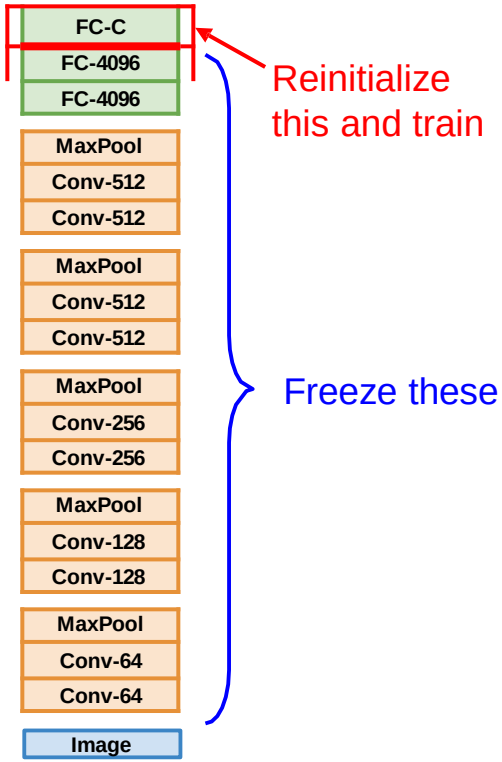
Transfer Learning with CNNs

Donahue et al, "DeCAF: A Deep Convolutional Activation Feature for Generic Visual Recognition", ICML 2014
Razavian et al, "CNN Features Off-the-Shelf: An Astounding Baseline for Recognition", CVPR Workshops 2014

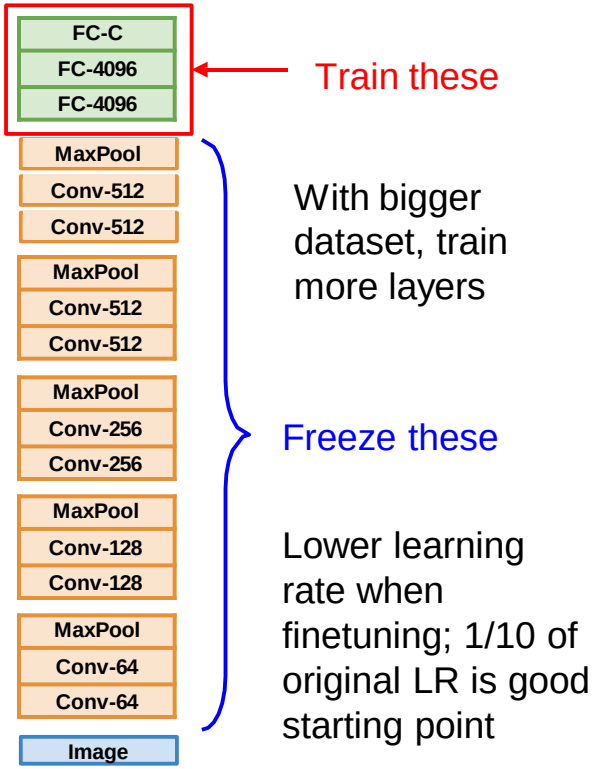
1. Train on Imagenet



2. Small Dataset (C classes)



3. Bigger dataset



Appendix

Xavier Initialization: Proof of Optimality

```
dims = [4096] * 7
hs = []
x = np.random.randn(16, dims[0])
for Din, Dout in zip(dims[:-1], dims[1:]):
    W = np.random.randn(Din, Dout) / np.sqrt(Din)
    x = np.tanh(x.dot(W))
    hs.append(x)
```

"Xavier" initialization:
std = 1/sqrt(Din)

"Just right": Activations are nicely scaled for all layers!
For conv layers, Din is $\text{filter_size}^2 * \text{input_channels}$

Let: $y = x_1 W_1 + x_2 W_2 + \ldots + x_{Din} W_{Din}$

Glorot and Bengio, "Understanding the difficulty of training deep feedforward neural networks", AISTAT 2010

Weight Initialization: "Xavier" Initialization

```
dims = [4096] * 7
hs = []
x = np.random.randn(16, dims[0])
for Din, Dout in zip(dims[:-1], dims[1:]):
    W = np.random.randn(Din, Dout) / np.sqrt(Din)
    x = np.tanh(x.dot(W))
    hs.append(x)
```

"Xavier" initialization:
std = 1/sqrt(Din)

"Just right": Activations are nicely scaled for all layers!
For conv layers, Din is $\text{filter_size}^2 * \text{input_channels}$

Let: $y = x_1w_1+x_2w_2+\ldots+x_{Din}w_{Din}$

Assume: $Var(x_1) = Var(x_2)= \ldots=Var(x_{Din})$

Glorot and Bengio, "Understanding the difficulty of training deep feedforward neural networks", AISTAT 2010

Weight Initialization: “Xavier” Initialization

```
dims = [4096] * 7
hs = []
x = np.random.randn(16, dims[0])
for Din, Dout in zip(dims[:-1], dims[1:]):
    W = np.random.randn(Din, Dout) / np.sqrt(Din)
    x = np.tanh(x.dot(W))
    hs.append(x)
```

“Xavier” initialization:
std = 1/sqrt(Din)

“Just right”: Activations are nicely scaled for all layers!
For conv layers, Din is $\text{filter_size}^2 * \text{input_channels}$

Let: $y = x_1w_1+x_2w_2+\ldots+x_{Din}w_{Din}$

Assume: $Var(x_1) = Var(x_2)= \ldots=Var(x_{Din})$

We want: $Var(y) = Var(x_i)$

Glorot and Bengio, “Understanding the difficulty of training deep feedforward neural networks”, AISTAT 2010

Weight Initialization: "Xavier" Initialization

```
dims = [4096] * 7
hs = []
x = np.random.randn(16, dims[0])
for Din, Dout in zip(dims[:-1], dims[1:]):
    W = np.random.randn(Din, Dout) / np.sqrt(Din)
    x = np.tanh(x.dot(W))
    hs.append(x)
```

"Xavier" initialization:
std = 1/sqrt(Din)

"Just right": Activations are nicely scaled for all layers!
For conv layers, Din is $\text{filter_size}^2 * \text{input_channels}$

Let: $y = x_1w_1+x_2w_2+\ldots+x_{Din}w_{Din}$

Assume: $Var(x_1) = Var(x_2) = \ldots = Var(x_{Din})$

We want: $Var(y) = Var(x_i)$

$Var(y) = Var(x_1w_1+x_2w_2+\ldots+x_{Din}w_{Din})$
[substituting value of y]

Glorot and Bengio, "Understanding the difficulty of training deep feedforward neural networks", AISTAT 2010

Weight Initialization: “Xavier” Initialization

```
dims = [4096] * 7
hs = []
x = np.random.randn(16, dims[0])
for Din, Dout in zip(dims[:-1], dims[1:]):
    W = np.random.randn(Din, Dout) / np.sqrt(Din)
    x = np.tanh(x.dot(W))
    hs.append(x)
```

“Xavier” initialization:
std = 1/sqrt(Din)

“Just right”: Activations are nicely scaled for all layers!
For conv layers, Din is $\text{filter_size}^2 * \text{input_channels}$

Let: $y = x_1w_1+x_2w_2+\ldots+x_{Din}w_{Din}$

Assume: $Var(x_1) = Var(x_2)= \ldots=Var(x_{Din})$

We want: $Var(y) = Var(x_i)$

$Var(y) = Var(x_1w_1+x_2w_2+\ldots+x_{Din}w_{Din})$
$= Din\ Var(x_iw_i)$
[Assume all x_i, w_i are iid]

Glorot and Bengio, “Understanding the difficulty of training deep feedforward neural networks”, AISTAT 2010

Weight Initialization: "Xavier" Initialization

```
dims = [4096] * 7
hs = []
x = np.random.randn(16, dims[0])
for Din, Dout in zip(dims[:-1], dims[1:]):
    W = np.random.randn(Din, Dout) / np.sqrt(Din)
    x = np.tanh(x.dot(W))
    hs.append(x)
```

"Xavier" initialization:
std = 1/sqrt(Din)

"Just right": Activations are nicely scaled for all layers!

For conv layers, Din is filter_size^2 * input_channels

Let: $y = x_1w_1+x_2w_2+\ldots+x_{Din}w_{Din}$

Assume: $Var(x_1) = Var(x_2) = \ldots = Var(x_{Din})$

We want: $Var(y) = Var(x_i)$

$Var(y) = Var(x_1w_1+x_2w_2+\ldots+x_{Din}w_{Din})$
$= Din\ Var(x_iw_i)$
$= Din\ Var(x_i)\ Var(w_i)$
[Assume all x_i, w_i are zero mean]

Glorot and Bengio, "Understanding the difficulty of training deep feedforward neural networks", AISTAT 2010

Weight Initialization: "Xavier" Initialization

```
dims = [4096] * 7
hs = []
x = np.random.randn(16, dims[0])
for Din, Dout in zip(dims[:-1], dims[1:]):
    W = np.random.randn(Din, Dout) / np.sqrt(Din)
    x = np.tanh(x.dot(W))
    hs.append(x)
```

"Xavier" initialization:
std = 1/sqrt(Din)

"Just right": Activations are nicely scaled for all layers!

For conv layers, Din is $\text{filter_size}^2 * \text{input_channels}$

Let: $y = x_1w_1+x_2w_2+\ldots+x_{Din}w_{Din}$

Assume: $Var(x_1) = Var(x_2) = \ldots = Var(x_{Din})$

We want: $Var(y) = Var(x_i)$

$Var(y) = Var(x_1w_1+x_2w_2+\ldots+x_{Din}w_{Din})$
$= Din\ Var(x_iw_i)$
$= Din\ Var(x_i)\ Var(w_i)$
[Assume all x_i, w_i are iid]

So, $Var(y) = Var(x_i)$ only when $Var(w_i) = 1/Din$

Glorot and Bengio, "Understanding the difficulty of training deep feedforward neural networks", AISTAT 2010